AY2025 Master's Thesis

Enhancing CryptOpt with Analysis and an LLVM Bridge

A Thesis Submitted to the Department of Computer Science and Communications Engineering, the Graduate School of Fundamental Science and Engineering of Waseda University in Partial Fulfillment of the Requirements for the Degree of Master of Engineering

Haruto Kimura

The Department of Computer Science and Communications Engineering, the Graduate School of Fundamental Science and Engineering of Waseda University

5123F031-5

Submission Date : 26th January, 2026

Research Guidance: : Research on Cryptographic Protocols

Advisor: : Prof. Kazue Sako

Abstract

High-speed cryptographic software is greatly desirable, yet off-the-shelf compilers often fail to match the performance of hand-tuned assembly. To address this, CryptOpt (Kuepper et al., PLDI 2023) employs evolutionary search to generate high-performance, constant-time assembly code. However, CryptOpt has restricted applicability due to its specific input format, and the relative effectiveness of its mutation strategies has not been empirically analyzed. This thesis enhances CryptOpt through three distinct contributions: analysis, expansion, and methodological validation. First, we analyze the optimizer's internal heuristics by isolating the effects of its two mutation strategies: instruction reordering (Permutation) and instruction selection (Decision). Our analysis reveals that Permutation mutations are the primary driver of performance improvements, while Decision mutations are more frequently accepted as new candidates during the optimization search. Second, we design and implement an LLVM Bridge that converts generic LLVM IR into CryptOpt's internal representation. Since LLVM IR is a standard target for languages like Rust and C, this expansion allows CryptOpt to optimize a broad range of cryptographic implementations beyond its original scope. Third, we evaluate the validity of the original CryptOpt performance measurements. We investigated whether the use of AssemblyLine, which executes code in in-memory buffers, could introduce artificial speedups compared to executing code loaded from standard shared objects used as baselines. Our re-evaluation indicates that AssemblyLine does not appear to yield an unfair performance gap. Furthermore, to rigorously evaluate code optimized by CryptOpt via our LLVM bridge without relying on comparisons against shared objects, we employ a direct assembly-to-assembly comparison framework. Finally, code optimized by CryptOpt via our LLVM bridge demonstrates performance improvements with a median speedup of 1.08 times (up to 1.39 times) using our assembly-to-assembly comparison and a median of 1.04 times (up to 1.60 times) with the original measurement framework over off-the-shelf compiler-generated code.

概要

暗号ソフトウェアを高速化するために最適化は必要である．汎用コンパイラは一定の最適化を行えるものの，ドメイン固有の最適化を十分に活用できず，専門家が手作業で書いたアセンブリほど高性能なコードを生成できない場合がある．これに対し CryptOpt (Kuepper ほか, PLDI 2023) は，進化的探索を用いることで，専門家が手書きで生成するアセンブリと同程度の性能を持つ定数時間で実行されるアセンブリコードを生成する．しかし，CryptOpt には入力フォーマットが限定されているという制約があり，また，その変異戦略の相対的な有効性は実証的に分析されていない．本論文では，CryptOpt を解析，拡張およびその性能測定の妥当性を検証するという三つの主要な側面で強化した．第一に，最適化の内部動作を解明するため，命令の並べ替え（Permutation）と命令の選択（Decision）という二つの変異戦略の影響を分離して評価した．分析の結果，Permutation 変異が性能向上の主要因である一方で, Decision 変異を適用した候補プログラムは，探索プロセスにおいて新たな候補としてより頻繁に採択されることを明らかにした．第二に，LLVM 中間表現（LLVM IR）を入力として受け取り，CryptOpt の最適化エンジンが扱える形式へ変換するコード変換手法（LLVM ブリッジ）を設計・実装した．LLVM IR は Rust や C 言語など多数のプログラミング言語で広く使用されているため，本拡張により CryptOpt は多様な言語のコードを最適化できるようになる．第三に，オリジナルの CryptOpt における性能測定の妥当性を評価した．具体的には，オリジナルの CryptOpt フレームワークが生成したコードをメモリ上で直接実行するために使用する AssemblyLine が，ベースラインとして用いられる標準的な共有オブジェクトをロードして実行する場合と比較して，見かけ上の高速化をもたらしていないかを検証した．再評価の結果，AssemblyLine は不公平な性能差を生じさせない可能性が高いことが示された．さらに，LLVM ブリッジを通じて CryptOpt により最適化されたコードの性能を厳密に評価するために，共有オブジェクトとの比較に依存しない，アセンブリコード同士を直接比較する新たな評価フレームワークを提案した．最終的に，LLVM ブリッジを通じて CryptOpt により最適化されたコードは，既存のコンパイラ生成コードと比較して，アセンブリ同士の比較で中央値 1.08 倍（最大 1.39 倍），オリジナルの測定フレームワークで中央値 1.04 倍（最大 1.60 倍）の性能向上を示した．

Contents

1 Introduction **8**

2 Background **12**
2.1 Finite Field Arithmetic Implementation 12
2.2 Relevant x86-64 Assembly Instructions 12
2.2.1 Carry-Propagating Arithmetic. 12
2.2.2 Flag-Preserving Operations. 12
2.2.3 Conditional Execution. 13
2.3 Fiat Cryptography 13
2.4 CryptOpt 13
2.4.1 Inputs and Scope 14
2.4.2 Optimization Strategy: Permutation and Decision 14
2.4.3 Measurement and Verification 15
2.5 Rust 15
2.6 LLVM 15
2.7 Timing Side Channels 16
2.8 CPU Microarchitecture: P-Core vs. E-Core 17

3 CryptOpt Mutation Analysis **18**
3.1 Analysis Methodology 18
3.2 Experimental Setup 18
3.3 Metric and Feature Tracking 18
3.4 Analysis Results 19
3.4.1 Performance Improvement by Mutation Ratio. 19
3.4.2 Mutation Success Rates. 21
3.4.3 Decision Mutation Category Analysis. 23
3.4.4 Permutation Distance Analysis. 26
3.5 Summary of Findings 29

4 LLVM Bridge **30**
4.1 Format Transformation 30
4.2 Bound Checks and Panic Blocks 31
4.3 Variable Renaming and Address Reconciliation 31
4.4 Extend Supported Instructions 31
4.4.1 Mapping SELECT to CMOVZNZ. 32
4.4.2 Mapping SEXT to CMOVZNZ. 32

5 Performance Evaluation of the LLVM Bridge **34**
5.1 Re-evaluation of Original CryptOpt 34
5.1.1 Re-evaluation Methodology. 34
5.1.2 Re-evaluation Results. 35
5.2 Evaluation of LLVM Bridge by our own comparison framework 36
5.2.1 Target Implementations 36

5.2.2 Comparison Framework 37
5.2.3 Experimental Setup 37
5.2.4 Performance Results 38
5.3 Evaluation of LLVM Bridge by MeasureSuite 42
5.3.1 Experimental Setup 42
5.3.2 Results 46
5.3.3 Comparison against Off-the-Shelf Compilers by MeasureSuite. 46

6 Discussion **49**
6.1 Performance Comparison Methodology 49
6.2 Analysis of the BLS12 Performance 49
6.3 Limitations with LLVM IR 50

7 Conclusion **51**

Acknowledgments **52**

References **53**

A Performance Measurement in CryptOpt **57**
A.1 Batching 57
A.2 Measurement with Dynamic Batch Size 59
A.3 Bet-and-Run Parameters 59

B CSS 2025 Paper **59**

List of Figures

1 Overview of the original CryptOpt workflow (simplified). 13
2 Performance improvement heatmap for multiplication 20
3 Performance improvement heatmap for squaring 20
4 Mutation success rates for multiplication 21
5 Mutation success rates for squaring 22
6 Correlation between success rate and performance 23
7 Decision mutation breakdown for multiplication 24
8 Decision mutation breakdown for squaring 25
9 Permutation success rates by distance for multiplication 27
10 Permutation success rates by distance for squaring 28
11 Overview of our framework. 30

List of Tables

1 Decision mutation category success rates 23
2 Re-evaluation of Original CryptOpt 35
3 LLVM bridge performance comparison on E-Core 39
4 Detailed statistics for Rust implementations (E-Core) 40
5 Detailed statistics for Fiat-Crypto C implementations (E-Core) 41
6 Detailed statistics for OpenSSL C implementations (E-Core) 42
7 LLVM bridge performance comparison on P-Core 43
8 Detailed statistics for Rust implementations (P-Core) 44
9 Detailed statistics for Fiat-Crypto C implementations (P-Core) 45
10 Detailed statistics for OpenSSL C implementations (P-Core) 46
11 LLVM bridge performance via MeasureSuite 47
12 Scalar multiplication measured with SUPERCOP (Best Cycles) 48

1 Introduction

Cryptography is a mechanism that offers protection for sensitive data. In this digital era, where cryptographic software is intensively and extensively used, there seems to be an endless demand for efficient implementations. The variety of cryptographic primitives, several options for security levels, and many possible parameters already pose challenges to implementers. Furthermore, since those implementations will eventually run on real devices, different hardware architectures as well as constraints on such devices make it even more challenging for implementers to produce secure and efficient cryptographic software.

One approach to handle portability is to implement cryptographic software in a high-level language, such as C or Rust, and rely on compilers to translate those programs into suitable machine code to run on the target platform. While this approach solves the issue of having to re-implement the software when the target platform changes (e.g., due to an introduction of newer generations of machines), such code performs slower than code produced by experts directly implementing in assembly [1, 2, 3, 4, 5, 6, 7, 8, 9]. However, a workflow that depends on a small pool of experts to hand-optimize assembly does not scale well with an increase in cryptographic software demands. Additionally, manual implementations may introduce some bugs.

On the aspect of verifying whether the implementations are indeed working or producing results as expected, there are various tools and techniques [10, 11, 12, 13, 14, 15, 16, 17, 18] that can be deployed. Similarly, there are many proposals to cope with the scalability of implementing and optimizing cryptographic code, some of which also integrate with formal verification. One example is SLOTHY [19], which is an automated tool that employs constraint solving to optimize assembly code for Arm architectures (AArch64).

Another example is CryptOpt [20], which is also an automated tool but employs an evolutionary algorithm for the optimization process. While CryptOpt [20] successfully tackles the problems of efficiency, security, and verification, it currently faces two limitations.

First, the internal dynamics of CryptOpt's optimization process remain opaque. The optimizer functions as a "black box" that stochastically applies mutations—specifically instruction reordering (permutation) and instruction selection (decision)—without a clear understanding of which strategy drives performance gains. This lack of insight hinders the development of more efficient search heuristics.

Second, the use case of CryptOpt is restricted by its specific input requirements. To be more precise, CryptOpt is tightly coupled to Fiat Cryptography [21] (a framework that translates functional programs into code with a proof of equivalence) and mainly accepts Fiat JSON, the JSON IR produced by Fiat Cryptography, as the input. This generally implies that CryptOpt is unable to optimize code that does not comply with the Fiat format. Even though CryptOpt explores inputs beyond Fiat Cryptography, to the best of our knowledge there is only one published case study on a non-Fiat implementation (Bitcoin Core secp256k1) [20].

Finally, valid performance measurement is critical for automated code generation. The original CryptOpt framework relies on a tool called AssemblyLine to assemble and execute generated code in an in-memory buffer, whereas baselines are loaded as shared objects. This distinction raised the question of whether the reported speedups were partially artifacts of the measurement harness (e.g., due to memory layout or loading differences) rather than code quality. To address this, we conducted a rigorous re-evaluation comparing in-memory execution against standard object compilation. Contrary

to potential concerns, our investigation indicates that AssemblyLine does not appear to introduce a significant performance gap. This finding not only validates the original CryptOpt results but also establishes a solid baseline for evaluating our new extensions.

To overcome this limitation and deepen the understanding of the optimization process, this work presents a threefold enhancement to CryptOpt: *analysis*, *expansion*, and *methodological validation*. First, regarding analysis, we dissect the internal evolutionary mechanisms of CryptOpt. By isolating the effects of instruction scheduling (Permutation) and instruction selection (Decision), we offer a detailed look at how each of them individually contributes to CryptOpt's optimization performance.

Second, regarding expansion, we design and implement a translation layer named the LLVM bridge. This bridge transforms generic LLVM intermediate representation (LLVM IR) —derived from languages like Rust and C—into Fiat JSON format, thereby decoupling CryptOpt from Fiat Cryptography and enabling it to optimize a broad range of cryptographic implementations.

Third, regarding methodological validation, we examine the performance measurement framework used in the original CryptOpt. We investigate whether the use of AssemblyLine, which executes code in in-memory buffers, introduces artificial speedups compared to standard shared object loading. Our re-evaluation indicates that AssemblyLine does not appear to yield an unfair performance gap, validating the accuracy of the original benchmarks. Additionally, we employ a direct assembly-to-assembly comparison framework to rigorously evaluate our new LLVM bridge.

To evaluate the effectiveness of our approach, we compare the performance of code optimized by CryptOpt via our LLVM bridge against code generated by off-the-shelf compilers. The results show that across 13 implementations (25 routines; multiplication and squaring per implementation; BLS12: multiplication only), code generated by CryptOpt via our LLVM bridge outperforms compiler-generated baselines in most cases, achieving a median speedup of 1.08 times (up to 1.39 times) using our assembly-to-assembly comparison and a median of 1.04 times (up to 1.60 times) with the original measurement framework compared to compiler-generated code. As a by-product, and thanks to the integration with CryptOpt, the code produced is also secure against timing attacks.

Our Contributions

In summary, this thesis makes the following contributions:

- *Analysis: Optimization Dynamics.* We analyze the efficacy of CryptOpt's mutation strategies by isolating their individual effects. We provide an empirical analysis of *Permutation* (scheduling) versus *Decision* (instruction selection) strategies. We identify that while Decision mutations have a higher success rate (26%) compared to Permutation (19%), a higher permutation ratio tends to yield better performance (Chapter 3).

- *Expansion: LLVM bridge.* We present a front-end that converts arbitrary LLVM intermediate representation (LLVM IR)—as emitted by common front-ends such as *clang* or *rustc*—into the JSON dialect expected by CryptOpt. This bridges the gap between high-level languages and CryptOpt's domain-specific optimizer (Chapter 4).

- *Evaluation: Methodological Validation and Benchmarking.* We examine the performance measurement methodology of the original framework. By comparing AssemblyLine-based measurements against an assembly-to-assembly approach, our results indicate that the original in-memory execution model does not appear to provide an unfair advantage, supporting the validity

of previous results. Along with this validated insight, we benchmark our new LLVM Bridge on seven real-world primitives and demonstrate a median speedup of 1.08 times (up to 1.39 times) using our assembly-to-assembly comparison and a median of 1.04 times (up to 1.60 times) with the original measurement framework over -O3 compiler output (Chapter 5).

Thesis Organization

This thesis is organized as follows:

Chapter 2: Background

This chapter provides the background of the thesis. It introduces finite field arithmetic implementation and the relevant x86-64 assembly instructions for this work, such as carry-propagating arithmetic and flag-preserving operations. Further, the chapter explains the existing frameworks Fiat Cryptography and CryptOpt. Moreover, it provides an overview of the Rust programming language and the LLVM compiler infrastructure, which are central to the thesis's expansion of CryptOpt. The chapter concludes with a discussion on timing side-channel attacks and modern CPU microarchitectures.

Chapter 3: CryptOpt Mutation Analysis

This chapter provides an in-depth analysis of the mutation strategies within CryptOpt. It dissects the optimizer's internal mechanics, specifically focusing on the two mutation operators: Permutation (instruction scheduling) and Decision (instruction selection). The chapter evaluates the contribution of each strategy to the overall performance improvement and analyzes the success rates of different mutation categories. It also examines the impact of swap distance in permutation mutations and each decision mutation's success rate to understand the optimizer's behavior.

Chapter 4: LLVM Bridge

This chapter describes the design and implementation of the LLVM bridge, a key contribution that extends CryptOpt's applicability. It details the transformation pipeline that converts generic LLVM IR from high-level languages like Rust and C into CryptOpt's internal representation. The chapter elaborates on technical challenges such as handling compiler-inserted panic blocks, variable renaming, reconciling addressing conventions, and mapping unsupported LLVM instructions like SELECT and SEXT to CryptOpt-compatible equivalents.

Chapter 5: Performance Evaluation of the LLVM Bridge

This chapter presents the performance evaluation of the LLVM Bridge. It first investigates the validity of the original CryptOpt performance measurement methodology, indicating that the use of in-memory execution via AssemblyLine does not appear to introduce artificial speedups. Then, it evaluates the performance of the code generated through the new LLVM bridge, comparing it against code produced by off-the-shelf compilers using our own comparison framework as well as the original CryptOpt framework across various cryptographic primitives and hardware platforms.

Chapter 6: Discussion

This chapter discusses the findings and limitations of the work. It validates the performance comparison methodology and analyzes specific performance behaviors, such as the code size increase in BLS12 due to register spilling. Finally, the chapter outlines current limitations regarding unsupported operations like ADDCARRYX and SUBBORROW that prevent certain implementations from being processed.

Chapter 7: Conclusion

This chapter summarizes the thesis and outlines potential directions for future work.

2 Background

This section explains concepts and previous works that are relevant to our work. We first describe finite field arithmetic implementation and relevant x86-64 assembly instructions, followed by two frameworks to automatically generate cryptographic code: Fiat Cryptography and CryptOpt. Then we discuss Rust and LLVM, including their significance in the compilation process. Finally, we provide background on constant-time programming and CPU microarchitecture about P-core and E-core.

2.1 Finite Field Arithmetic Implementation

Public-key cryptography, such as Elliptic Curve Cryptography (ECC), relies heavily on arithmetic operations over finite fields [22]. The elements of these fields are often large integers, typically ranging from 255 to 381 bits or more. The core operations required are addition, subtraction, multiplication, squaring, and modular reduction modulo a large prime p. Since these integers are much larger than the 64-bit registers found in modern CPUs, they must be represented as arrays of smaller machine words (limbs). Performing arithmetic on these multi-limb representations requires careful handling of carries and borrows between limbs, making these operations computationally expensive and a prime target for optimization.

2.2 Relevant x86-64 Assembly Instructions

Modern cryptographic optimizations rely on choosing the right assembly instructions to perform mathematical operations on large integers (much larger than the CPU's native 64-bit registers). CryptOpt generates x86-64 assembly code by exploiting specific features of the x86-64 instruction set to maximize performance [23, 24].

2.2.1 Carry-Propagating Arithmetic.

When adding two large numbers that are split into smaller 64-bit parts (limbs), the *carry* from one addition must be added to the next part. Traditionally, x86-64 processors use a single flag (the Carry Flag, CF) to track this carry. The ADD instruction starts the chain, and ADC (Add with Carry) continues it. Because there is only one Carry Flag, these additions must happen one after another in a strict sequence. The Intel ADX extension introduced two new instructions: ADCX, which uses the Carry Flag, and ADOX, which uses the Overflow Flag (OF) to act as a second carry flag. This creates two parallel lanes for addition. CryptOpt can use these to calculate two parts of a large number sum simultaneously, speeding up the calculation.

2.2.2 Flag-Preserving Operations.

Most arithmetic instructions update the CPU flags to reflect their result. If we run such an instruction in the middle of a carry chain, it would overwrite the carry flag, breaking the calculation. CryptOpt prioritizes instructions that perform math *without* modifying flags. Standard multiplication instructions typically scramble the flags. MULX computes a 128-bit product from two 64-bit numbers without touching any flags. This means CryptOpt can insert multiplications right in the middle of a sequence of additions without needing to save and restore the flag state. Although designed for calculating

memory addresses, LEA is often used for simple math (like $x = a + b$) because it does not affect flags. CryptOpt uses this to perform fast additions or small multiplications safely within critical code sections.

2.2.3 Conditional Execution.

To prevent timing attacks, cryptographic code must not use secret data to decide which path to take. Instead of jumping, the CMOV instruction checks a flag and copies data only if the condition is met. This allows the CPU to execute the same sequence of instructions regardless of the secret data, keeping the execution time constant.

2.3 Fiat Cryptography

Fiat Cryptography [21] is a framework designed to automatically generate cryptographic code from mathematical specifications of cryptographic algorithms. In particular, the framework takes high-level formal specifications of field arithmetic, expressed as functional programs within the Coq proof assistant. Fiat Cryptography uses Coq to synthesize low-level code from these specifications while simultaneously proving that the generated code correctly implements the initial mathematical definitions. This process typically involves generating an intermediate representation, known as Fiat IR, which captures the arithmetic operations. From Fiat IR, various backends can produce code in target languages such as C, Rust, or Zig.

2.4 CryptOpt

CryptOpt [20] is a code generator designed to produce highly optimized x86-64 assembly code for cryptographic primitives, whose optimizations are tailored to the specific processor architecture for which they are expected to run. Additionally, CryptOpt guarantees that the optimized code it produces is correct, thanks to the integration with the Fiat Cryptography framework.

Figure 1 outlines the connection between Fiat Cryptography and CryptOpt frameworks. CryptOpt takes Fiat IR, a low-level intermediate representation produced by Fiat Cryptography, as input. Then it produces a JSON-structured representation of the Fiat IR that captures all relevant details such as operations, input/output widths, and bounds. This JSON is then used in CryptOpt's optimizer to produce assembly code. CryptOpt also integrates with a performance measurement tool MeasureSuite, which takes multiple pieces of input code and compares their execution time.

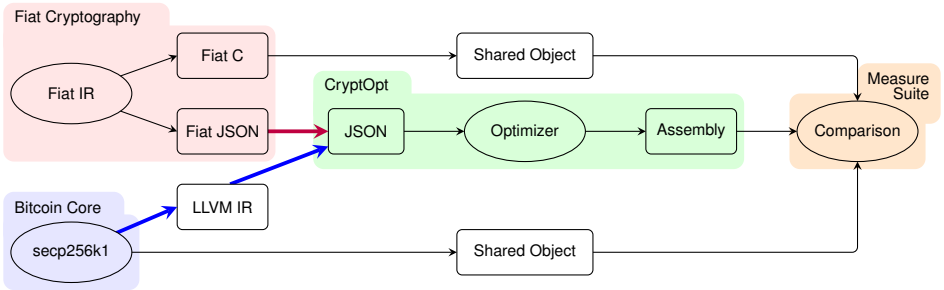

Figure 1: Overview of the original CryptOpt workflow (simplified).

2.4.1 Inputs and Scope

While CryptOpt generally takes Fiat IR as input, CryptOpt can also accept another type of input. The blue arrows in the bottom left of Figure 1 show such an option. In particular, CryptOpt can receive as input an LLVM IR generated from a Bitcoin Core implementation. We refer to this workflow as the *Bitcoin Core bridge*. Note that this workflow supports only the multiply and square functions on the Bitcoin Core secp256k1 curve implemented in C. This C implementation is first compiled to LLVM IR using *Clang*, a C/C++ compiler front end. Relevant information, such as LLVM IR operations, is then extracted and converted into an intermediate JSON format suitable for CryptOpt. This JSON captures essential operations, in the same structure as the JSON obtained from Fiat IR, but is derived directly from LLVM IR. This JSON is further translated into CryptOpt's internal representation for optimization.

2.4.2 Optimization Strategy: Permutation and Decision

CryptOpt employs a Random Local Search (RLS) [25] algorithm that iteratively mutates a candidate program. To ensure that these random mutations do not break the program's correctness, CryptOpt operates on an internal Data-Flow Graph (DFG) where nodes represent operations and edges represent data dependencies. The search space is explored using two distinct mutation operators: *Permutation* (M_P) and *Decision* (M_D).

Permutation (M_P): Instruction Scheduling. The Permutation mutation (M_P) optimizes the order of instructions to better suit the CPU's processing pipeline. It does not simply shuffle instructions randomly; instead, it respects the data dependencies between instructions by topologically sorting the DFG. For every instruction, CryptOpt identifies a valid range of positions. It must occur after its inputs are produced (producers) and before its result is consumed (consumers). Moving instructions within this range offers two performance benefits. First, placing a consumer closer to its producer reduces the time a value occupies a register. This minimizes *register pressure*, preventing the CPU from running out of fast registers and having to use slow memory (stack spilling). Second, mixing independent instructions allows the CPU to execute them simultaneously (instruction-level parallelism), preventing the processor from stalling while waiting for long operations to complete.

Decision (M_D): Instruction Selection. The Decision mutation (M_D) optimizes the specific machine instructions used to implement an operation. Just as there are multiple ways to phrase a sentence, there are multiple assembly instructions to perform the same mathematical calculation. One key optimization is *Carry Chain Selection* for multi-limb addition. Standard addition relies on a single Carry Flag, forcing additions to happen one after another. M_D can enable the use of modern instructions (ADCX and ADOX) that utilize two separate flags (Carry and Overflow). This allows two independent chains of additions to be processed in parallel. Another optimization is *Strength Reduction*. For simpler operations, such as multiplying by a small constant, M_D replaces heavy instructions like IMUL with lighter, faster alternatives like SHL (shift) or LEA (load effective address). Finally, M_D performs *Operand Selection* for commutative operations. By intelligently swapping the order of operands to match where data is currently stored, it eliminates unnecessary data copying (MOV instructions).

2.4.3 Measurement and Verification

Additionally, CryptOpt directly measures the execution time of both the original and mutated versions on the target hardware. CryptOpt keeps the faster version, effectively searching the space of possible machine-code implementations for the best one within a given computational budget. This empirical approach allows CryptOpt to automatically adapt to new hardware architectures and microarchitectural details.

For the proof of correctness, CryptOpt achieves this through a program-equivalence checker, which is integrated in MeasureSuite [26], a performance measurement tool that CryptOpt utilizes. In brief, MeasureSuite receives a set of input implementations, which can be in different formats, e.g., shared object (`.so`), object file (`.o`), or Intel-syntax NASM assembly (`.asm`). When handling NASM inputs, specifically the candidate code generated by CryptOpt, MeasureSuite relies on AssemblyLine [27]. AssemblyLine is an ultra-lightweight C library designed for the dynamic generation of x86-64 machine code without the overhead of invoking external compilers, assemblers, or linkers. It supports modern instruction sets required by cryptographic primitives, such as MMX, SSE2, AVX, and AVX2. Crucially for CryptOpt's rapid "mutate-assemble-measure" optimization cycle, AssemblyLine facilitates direct memory management; it writes machine code directly to user-allocated buffers (e.g., via `mmap` with `PROT_EXEC` permissions) or internally managed memory. This allows MeasureSuite to assemble the NASM assembly into an in-memory executable buffer on the fly, which is then executed for performance comparison and verified for functional equivalence through a bit-by-bit check.

2.5 Rust

Rust is a modern systems programming language known for its strong focus on performance, reliability, and memory safety. It achieves memory safety without requiring a garbage collector by employing a sophisticated type system and ownership model with compile-time checks [28]. These features prevent common programming errors such as null pointer dereferences, buffer overflows, and data races. Eliminating such errors is crucially important, particularly in security-sensitive cryptographic software.

Furthermore, Rust provides low-level controls comparable to C and C++, allowing developers to write highly performant code. Its rich ecosystem, managed by the *Cargo* package manager and build system, facilitates the development and sharing of libraries called *crates*. These characteristics combine the safety guarantees of high-level languages with the performance of lower-level code. Hence, there has been an increase in Rust adoption for implementing cryptographic protocols and primitives. In fact, Rust has been adopted for performance and security-critical components such as the rustls TLS stack and other libraries in RustCrypto project [29, 30].

The standard Rust compiler, *rustc*, uses LLVM as its backend. This means that Rust code is typically compiled down to LLVM intermediate representation before being translated into machine code. This shared compilation infrastructure with languages such as C and C++ via *Clang* provides a common ground for analysis and generalization [31].

2.6 LLVM

The LLVM Project [32] is a collection of modular and reusable compiler and toolchain technologies. Its core component relevant to this work is the LLVM intermediate representation (or LLVM IR for

short). LLVM IR is a low-level, language-independent representation designed as a target for compiler frontends and as an input for optimization passes and code generation backends.

LLVM IR uses a static single assignment (SSA) form, meaning each register is assigned exactly once. This helps simplify many analyses and transformations. Compilers for various high-level languages, including *Clang* for C/C++ and *rustc* for Rust, can generate LLVM IR from source code. LLVM then provides a suite of optimization passes that operate on this common IR. Finally, LLVM backends translate the optimized IR into machine code for diverse target architectures, such as x86-64.

The existence of LLVM IR as a common target for multiple source languages is crucial for generalization. By compiling both Rust and C code to LLVM IR, it is possible to develop a single transformation pipeline that converts this IR into a format suitable for a target use (e.g., to integrate with CryptOpt), rather than needing a separate handling for each source language.

2.7 Timing Side Channels

When executing cryptographic code on real-world devices, the interaction between the running device and the execution environment may leak *side-channel* information, such as execution time [33, 34, 35], power consumption [36, 37, 38], and electromagnetic radiation [39, 40]. Depending on the information leaked, performing side-channel analysis [33, 36] could potentially reveal the underlying cryptographic keys.

Timing side-channel attacks are known to be powerful (i.e., can recover the entire secret keys) and relatively easy to perform (i.e., capturing and comparing the execution time). If an attacker could observe timing variations of secret-dependent operations, such relationships would enable an attacker to extract the underlying secret. A fundamental principle in mitigating timing side-channel attacks is to ensure that a program's execution time is independent of any secret values it processes. A common coding practice against timing side-channel attacks is to write constant-time code, which basically avoids (1) secret-dependent control flows, (2) secret-dependent memory accesses, and (3) secret-dependent instructions.

There exist several tools [41] to help developers check their code for constant-timeness. One of them is dudect [42], an open-source empirical testing framework for detecting secret-dependent execution-time variations. This tool executes the target routine thousands of times on two input distributions, a fixed pattern (often all zeros) and a randomly generated pattern, then records the cycle counts of each invocation. The two timing distributions are then compared with Welch's two-sample t-test [43], which basically checks whether the means of two independent distributions differ, using the following formula:

$$t = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{s_1^2}{N_1} + \frac{s_2^2}{N_2}}}$$

where $\bar{X}_i$, s_i^2, and N_i denote the mean, variance, and sample size of timing set $i \in \{1, 2\}$. A routine is flagged as `FAIL` if $|t| > 4.5$. Passing the test, therefore, offers an empirical assurance that no secret-dependent timing variation is detectable.

2.8 CPU Microarchitecture: P-Core vs. E-Core

Intel's hybrid architecture, introduced with the 12th Generation and continued in Raptor Lake processors, combines high-performance Performance-cores (P-cores) and energy-efficient Efficient-cores (E-cores) [44]. P-cores deliver high single-thread performance and low latency, making them suitable for complex workloads and foreground tasks that directly impact user experience [45]. In contrast, E-cores offer higher energy efficiency and excellent multi-threaded throughput, which makes them ideal for background work, parallel workloads, and tasks marked with lower Quality-of-Service (QoS) [44, 45]. Intel Thread Director further enhances this design by providing the operating system with fine-grained runtime telemetry, allowing threads to be scheduled dynamically onto the most appropriate core type [44].

3 CryptOpt Mutation Analysis

To better understand what actually drives the performance gains reported by CryptOpt [20], this section takes a closer look at how its optimization procedure behaves internally. As described earlier in Chapter 2.4, the optimizer relies on two kinds of mutations: *Permutation (M_P)* for reordering instructions and *Decision (M_D)* for choosing between alternative instruction forms. In the default configuration, CryptOpt splits its choices evenly between the two ($M_P : M_D = 50 : 50$). However, it remains unclear which of these strategies is primarily responsible for the resulting speedups. To address this, we performed a more fine-grained analysis of how effective each mutation type actually is.

3.1 Analysis Methodology

An experiment was configured across 13 distinct cryptographic curves (including Curve25519, P-256, and secp256k1) for both multiplication and squaring routines from Fiat Cryptography [21] and AU-Curves [46]. For each routine, the mutation probability ratio M_P:M_D was varied from $100 : 0$ (pure permutation) to $0 : 100$ (pure decision) in increments of 10%, resulting in 11 distinct configurations per primitive. Each configuration was executed with a total budget of 10,000 mutations using a Bet-and-Run strategy (20 bets $\times$ 50 mutations, followed by 9,000 mutations on the best candidate).

3.2 Experimental Setup

All experiments were conducted on a 13th Gen Intel Core i7-1360P (Raptor Lake-P) running Ubuntu 22.04.4 LTS. As our experiments focus on the optimization process, we followed the original CryptOpt [20] setup during the optimization process. We pinned each optimization run to P-cores to reduce noise from context switching.

3.3 Metric and Feature Tracking

Beyond standard performance comparisons, the optimizer was customized to record detailed information about each mutation. For every attempted mutation, two primary performance indicators were measured. First, the Success Rate, defined as the proportion of mutations that the optimization process accepts into the current candidate program. Second, the Improvement Rate, which captures the proportion of mutations that reduce the measured cycle count.

To understand how the optimizer explores and exploits the search space, structural features of each mutation were further analyzed. For Permutation mutations (M_P), the swap distance $|d|$ between the two instructions involved was recorded. Rather than treating all reorderings uniformly, these distances were grouped into three categories (short ($|d| < 5$), medium ($5 \leq |d| < 15$), and long ($|d| \geq 15$)) to characterize the locality of effective schedule adjustments and identify whether beneficial improvements arise predominantly from fine-grained or coarse instruction movements. For Decision mutations (M_D), each instance was classified based on the semantic nature of the change introduced. By referring to the original CryptOpt [20] implementation, six categories are distinguished as follows:

- *IM (Immediate Selection)*: Selects which constant value (`0x0` or `-0x1`) to use when clearing the Overflow Flag. Since x86-64 lacks a direct instruction to clear OF, the optimizer uses INC or DEC on a register containing these values, as these instructions modify OF without affecting CF.

- *AR (Argument Reordering)*: Selects operand ordering for commutative operations. Different orderings affect register allocation decisions, as the first operand typically becomes the destination in x86-64, potentially reducing register moves.
- *KK (Carry Handling)*: Resolves flag conflicts when both processor flags (CF and OF) are in an undefined state. The optimizer chooses between using a simple destructive addition or clearing flags first to enable ADX instructions, which support parallel carry chains.
- *FL (Flag Selection)*: Chooses between the Carry Flag (CF) and Overflow Flag (OF) for carry propagation. The ADX extension provides independent carry channels via ADCX (CF) and ADOX (OF), enabling parallel multi-limb arithmetic.
- *MU (Multiplication Method)*: Selects instruction encoding for constant multiplications among IMUL, SHL, SHLX, and LEA. These differ in latency, flag side-effects, and register requirements.
- *B& (AND Instruction)*: Chooses between AND and BZHI for bit masking. The BZHI instruction is applicable when the mask selects consecutive low bits, requiring only a bit count rather than the full mask value.

This categorization enables analysis of which decision classes contribute most to the performance improvements.

3.4 Analysis Results

A total of 286 experimental configurations were executed, covering 13 curves across both multiplication and squaring operations at 11 distinct M_P:M_D ratios. Each cell in the heatmaps (Figures 2 and 3) represents a single experimental run, showing the percentage improvement in cycle count against the baseline from C implementation of Fiat Cryptography [21] or AUCurves [46] compiled by Clang. The bar charts presented in subsequent subsections aggregate these results by computing the arithmetic mean of success rates across all 11 ratio configurations for each curve, thereby characterizing the typical behavior of each primitive independent of the specific M_P:M_D ratio used.

3.4.1 Performance Improvement by Mutation Ratio.

Figures 2 and 3 present the performance improvements achieved across all curve and ratio combinations for multiplication and squaring, respectively.

Across all 143 multiplication experiments (13 curves $\times$ 11 ratios), the mean improvement is 23.9%, with individual results ranging from a maximum of 62.3% (P-256 at M_P:M_D = 90:10) to a minimum of -22.1% (P-384 at M_P:M_D = 0:100). On the other hand, the 143 squaring experiments show more modest gains: a mean of 14.7%, a maximum of 49.7% (P-256 at M_P:M_D = 80:20), and a minimum of -25.8% (P-384 at M_P:M_D = 0:100). The performance gap arises because squaring operations have a simpler structure (multiplying a number by itself), leaving fewer inefficiencies for the optimizer to remove compared to general multiplication.

Also, the choice of M_P:M_D ratio can affect the optimization outcomes. Averaging across all curves and both operations, pure Decision configurations (M_P:M_D = 0:100) yield a mean improvement of only -0.6%, whereas pure Permutation configurations (M_P:M_D = 100:0) achieve 23.5%. The best

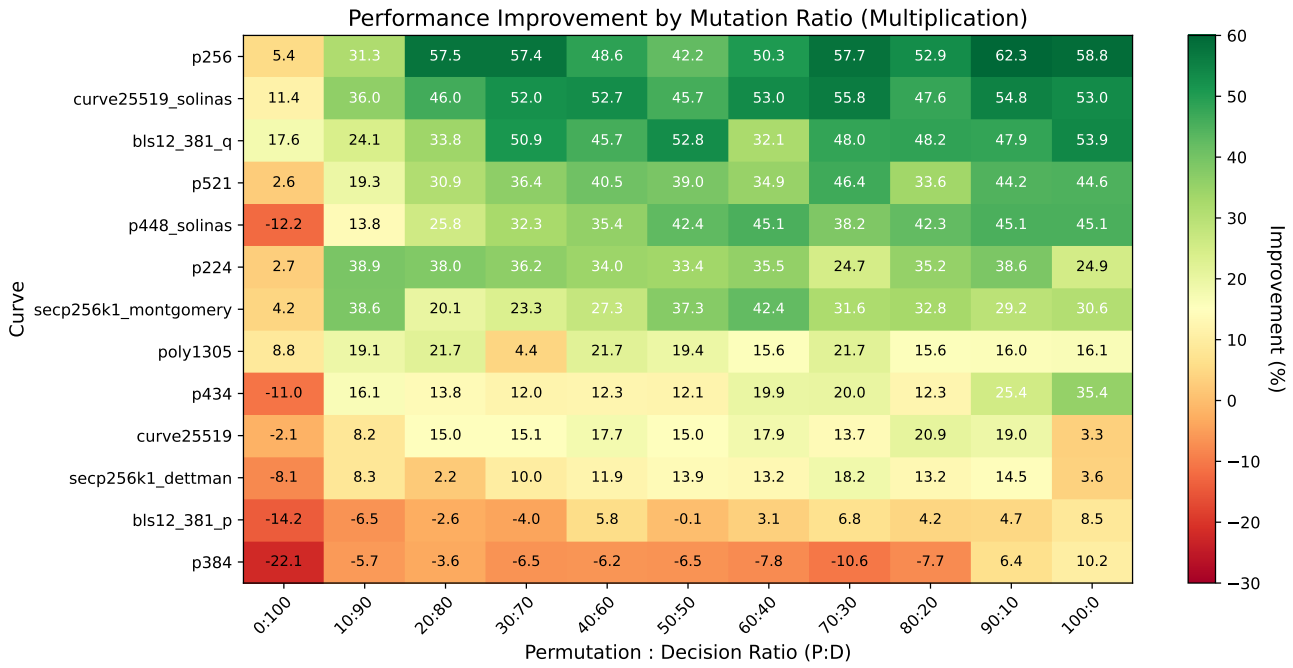

Figure 2: Performance improvement (%) for multiplication routines across curves and mutation ratios. Each cell shows the improvement for one curve at one M_P:M_D ratio. Curves are sorted by mean improvement across all ratios (descending). Green indicates improvement over the baseline; red indicates degradation.

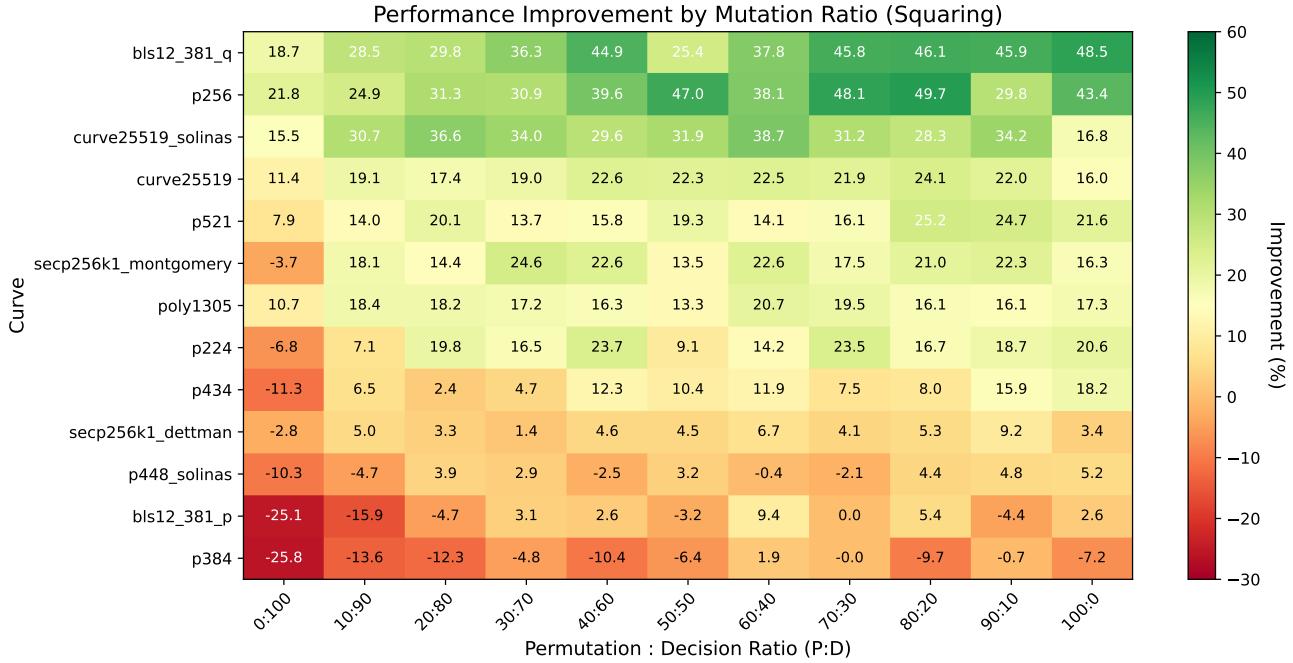

Figure 3: Performance improvement (%) for squaring routines across curves and mutation ratios. Curves are sorted by mean improvement across all ratios (descending).

average performance occurs at M_P:M_D = 90:10 (24.9%), with ratios from 70:30 through 100:0 all exceeding 22% mean improvement.

Therefore, this pattern indicates that Permutation mutations contribute more substantially to performance gains than Decision mutations, though some Decision mutations remain beneficial for exploring alternative code sequences.

3.4.2 Mutation Success Rates.

Figures 4 and 5 compare the success rates of Permutation and Decision mutations across curves. Each bar represents the mean success rate for that curve, computed by averaging the success rates recorded across all 11 M_P:M_D ratio configurations.

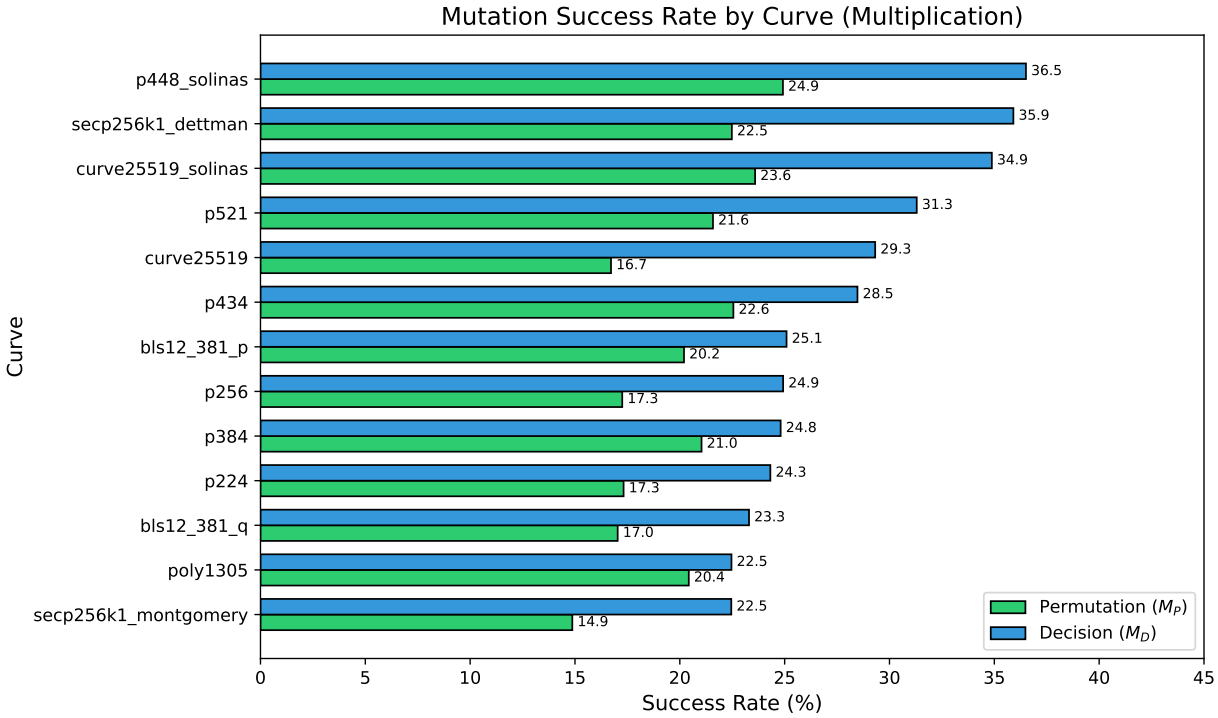

Figure 4: Mutation success rates for multiplication routines by curve. Each bar shows the mean success rate across all 11 M_P:M_D ratio configurations.

Among the 13 curves examined, Decision mutations achieve a mean success rate of 25.9%, compared to 18.8% for Permutation mutations. This difference is consistent across curves. In multiplication, Decision success rates range from 22.5% (poly1305, secp256k1_montgomery) to 36.5% (p448_solinas), while Permutation success rates range from 14.9% (secp256k1_montgomery) to 24.9% (p448_solinas). For every curve examined, Decision mutations are accepted approximately 1.3 to 1.6 times more frequently than Permutation mutations.

The higher acceptance rate of Decision mutations reflects the nature of the transformations involved. Instruction selection changes (e.g., swapping operand order, toggling carry flags) often preserve or slightly improve performance, which leads to acceptance under the stochastic hill-climbing

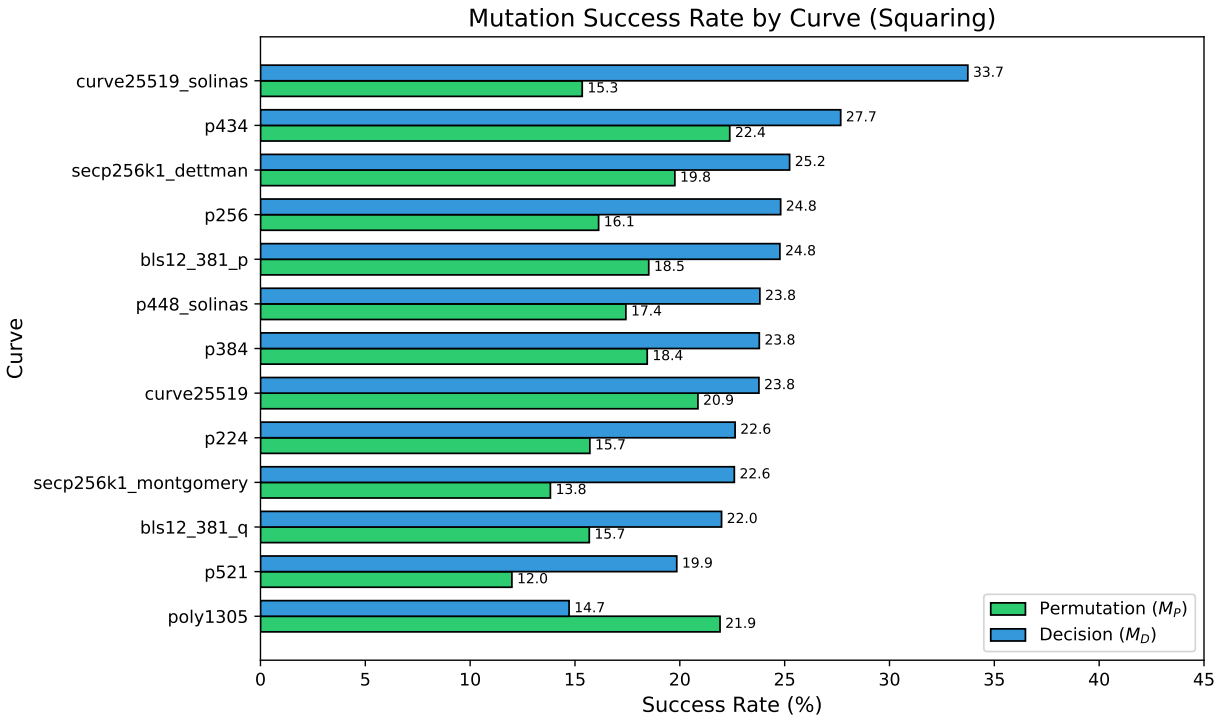

Figure 5: Mutation success rates for squaring routines by curve. Each bar shows the mean success rate across all 11 M_P:M_D ratio configurations.

criterion. Schedule perturbations, by contrast, more frequently increase register pressure since they blindly swap instructions as long as it satisfies the topological ordering constraint, causing rejection.

However, higher success rates do not directly translate to greater performance contributions. As established in Chapter 3.4.1, configurations dominated by Permutation mutations (M_P:$M_D \geq$ 70:30) yield superior final performance despite their lower per-mutation acceptance rates. This paradox arises because each successful Permutation mutation tends to decrease the cycle count by a larger margin whereas each successful Decision mutation typically yields only a small performance gain. The optimizer thus benefits from attempting many Permutation mutations even though fewer are accepted. Hence, this suggests that while Decision mutations are safer and easier to validate, they tend to result in local minima. Permutation mutations, on the other hand, are riskier with lower acceptance rates, but provide the structural changes necessary to escape these local minima and reach globally superior instruction schedules.

Correlation Analysis. To quantify whether mutation acceptance predicts final performance, we computed Pearson correlations between success rates and improvement across all 286 experiments. Figure 6 presents the results. Surprisingly, permutation success rate shows no correlation with final improvement ($r = -0.03$, $p = 0.66$), while decision success rate exhibits a moderate positive correlation ($r = 0.24$, $p < 0.001$). This counterintuitive finding reinforces our earlier observation: permutation mutations drive performance through *exploration*—enabling escape from local optima regardless of acceptance rate—while decision mutations succeed more readily on inherently easier optimization targets. The implication for practitioners is that low permutation acceptance should not be interpreted as

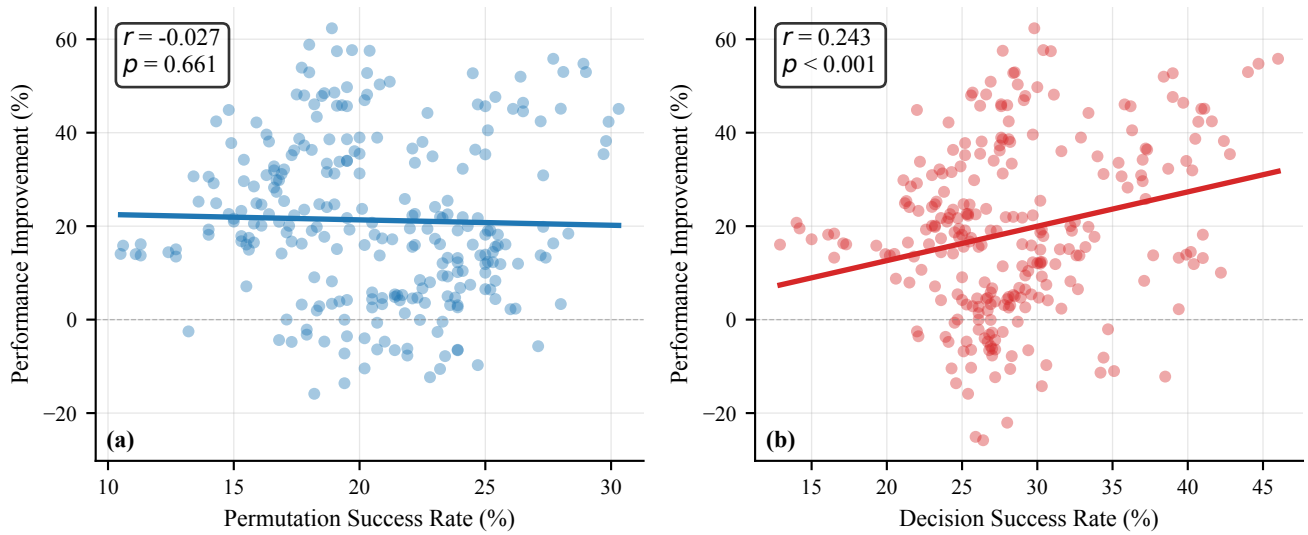

Figure 6: Correlation between mutation success rate and performance improvement. (a) Permutation success rate shows no correlation ($r = -0.03$, $p = 0.66$), suggesting permutation drives improvement through exploration rather than exploitation. (b) Decision success rate exhibits moderate positive correlation ($r = 0.24$, $p < 0.001$), indicating these mutations succeed more readily on easier optimization targets.

ineffective search; rather, it reflects the exploratory nature of structural code transformations.

3.4.3 Decision Mutation Category Analysis.

Figures 7 and 8 show the decomposition of Decision mutations' success rates by category. As with the previous figures (Figures 4 and 5), each bar represents the mean success rate for that category across all 11 M_P:M_D ratio configurations. The five categories—AR (argument reordering), KK (carry propagation), FL (flag handling), MU (immediate multiplication), and B& (bit masking)—correspond to distinct classes of code transformations within the Decision mutation operator. Also, Table 1 summarizes the mean success rates (%) for each Decision mutation category, computed across all experiments.

Table 1: Mean success rates (%) for Decision mutation categories. AR consistently achieves the highest success rate, while B& is the least effective. Note: MU and B& categories are only available for a subset of curves.

	IM	AR	FL	KK	MU	B&
Multiplication	21.9	30.9	19.8	17.9	13.3	8.8
Squaring	18.4	26.0	18.5	14.3	15.9	5.6
Overall	20.2	28.5	19.1	16.1	14.9	7.2

The high success rate of AR mutations is largely due to the commutative nature of many arithmetic operations. Swapping operand order in addition or multiplication preserves semantic correctness while

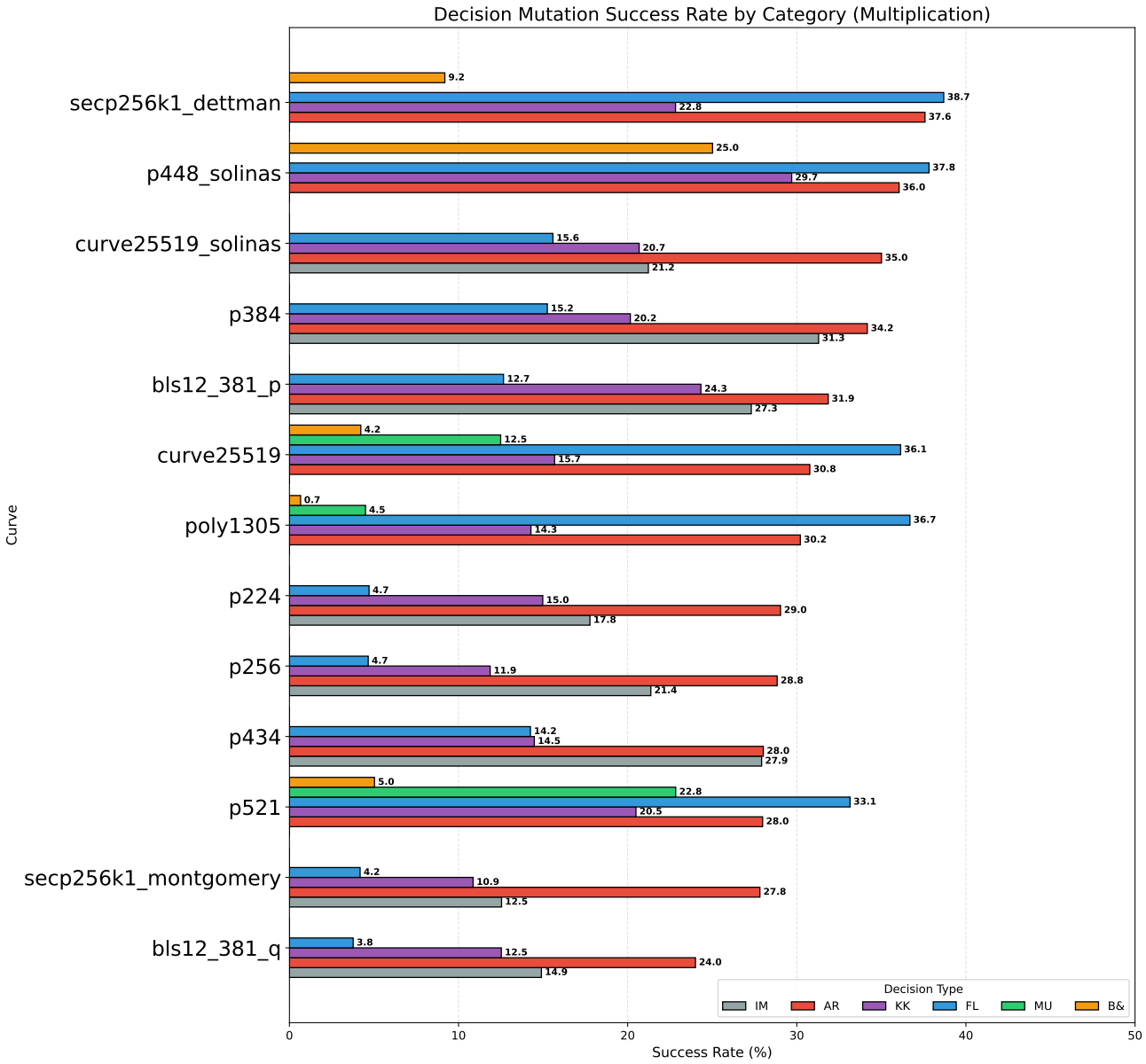

Figure 7: Decision mutation success rates by category for multiplication. Each bar shows the mean success rate for that category across all 11 M_P:M_D ratio configurations. AR: argument reordering, KK: carry propagation, FL: flag handling, MU: immediate multiplication, B&: bit masking.

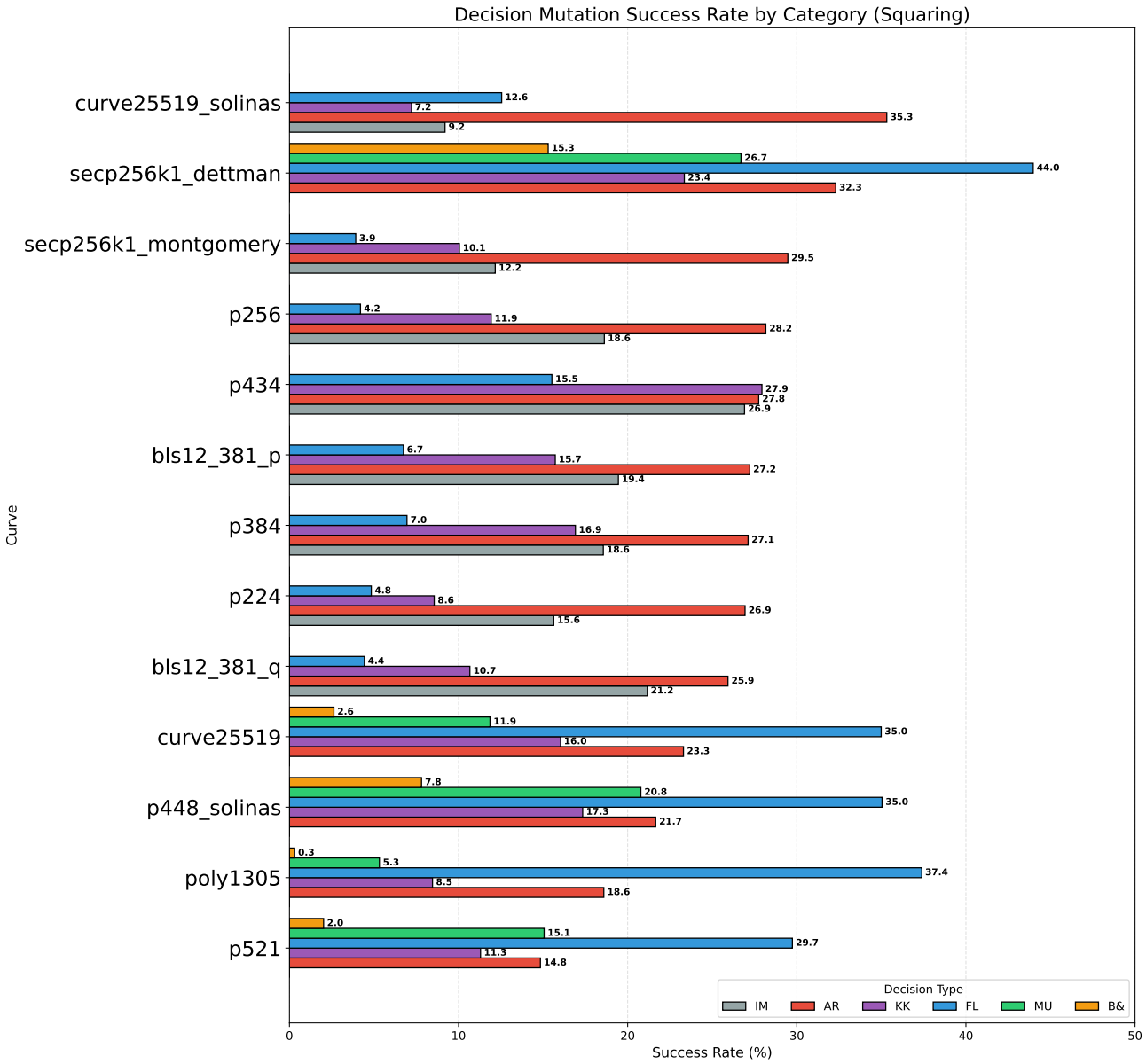

Figure 8: Decision mutation success rates by category for squaring. Each bar shows the mean success rate for that category across all 11 M_P:M_D ratio configurations.

potentially improving register allocation or enabling more favorable instruction scheduling. The relatively high success rate of FL mutations (19.1%) indicates that alternative flag-handling strategies such as using different condition codes or explicit comparisons, often yield equivalent or better performance.

On the other hand, the low success rate of B& mutations (7.2%) reflects the limited contexts in which AND and BZHI are interchangeable. The BZHI instruction, which extracts bits from a specified position, requires specific operand constraints that are not always satisfied in cryptographic code.

The low success rate of MU mutations (14.9%) is due to the choice among IMUL, SHL, SHLX, and LEA for constant multiplication depends on the specific constant value and available registers.

Notably, curves utilizing Solinas reduction exhibit higher B& and MU success rates than other curves. For multiplication, p448_solinas achieves 25.0% B& success, while the overall mean of 7.2% is achieved by Curve25519_solinas. For squaring, p448_solinas achieves 20.8% MU success and Curve25519_solinas achieves 12.7% MU success. This pattern stems from the sparse structure of Solinas primes, which involves multiplication by small constants (amenable to alternative MU strategies) and bit masking operations (amenable to B& alternatives).

We note that the overall Decision mutation success rate (26%) is not a simple average of the category-specific rates in Table 1. This is because the different mutation categories do not occur with equal frequency. For instance, AR mutations are applicable to a much larger number of instructions than B& or MU mutations, thus contributing more heavily to the aggregate success rate.

3.4.4 Permutation Distance Analysis.

Figures 9 and 10 examine the relationship between swap distance and Permutation mutation success rates. Swap distance $|d|$ is defined as the absolute difference in instruction indices between the two instructions being exchanged. As with previous figures, each bar represents the mean success rate across all 11 M_P:M_D ratio configurations for that curve.

Averaging across all 286 experiments, short-distance swaps ($|d| < 5$) achieve a mean success rate of 20.0%, medium-distance swaps ($5 \leq |d| < 15$) achieve 12.5%, and long-distance swaps ($|d| \geq 15$) achieve only 8.1%. This monotonic decrease in success rate with increasing swap distance is consistent across all curves. For multiplication, short-distance success rates range from 16.7% (bls12_381_q) to 29.3% (p448_solinas), while long-distance success rates range from 1.0% (poly1305) to 20.3% (p384).

The superior success rate of short-distance swaps can be attributed to data dependency constraints inherent in cryptographic code. Instructions that are close in the original schedule are more likely to be data-independent or have their dependencies satisfied by intermediate instructions, making their reordering feasible without violating program semantics. Long-distance swaps, on the other hand, frequently span producer-consumer pairs or introduce excessive register pressure, causing rejection.

The variation in long-distance success rates across curves correlates with code structure. Implementations like p384 and bls12_381_p, which use generic reduction algorithms with more uniform instruction patterns, exhibit relatively high long-distance success (15–21%) because their instruction sequences contain fewer tight dependency chains. Implementations like poly1305, which uses specialized reduction with dense carry propagation, show low long-distance success (1.0%) because most instructions depend closely on their predecessors.

Despite their lower acceptance rates, long-distance swaps that succeed may contribute disproportionately large performance improvements by breaking suboptimal instruction groupings inherited from the input IR. The optimization process thus benefits from attempting such swaps as part of its exploration strategy, even when most are rejected.

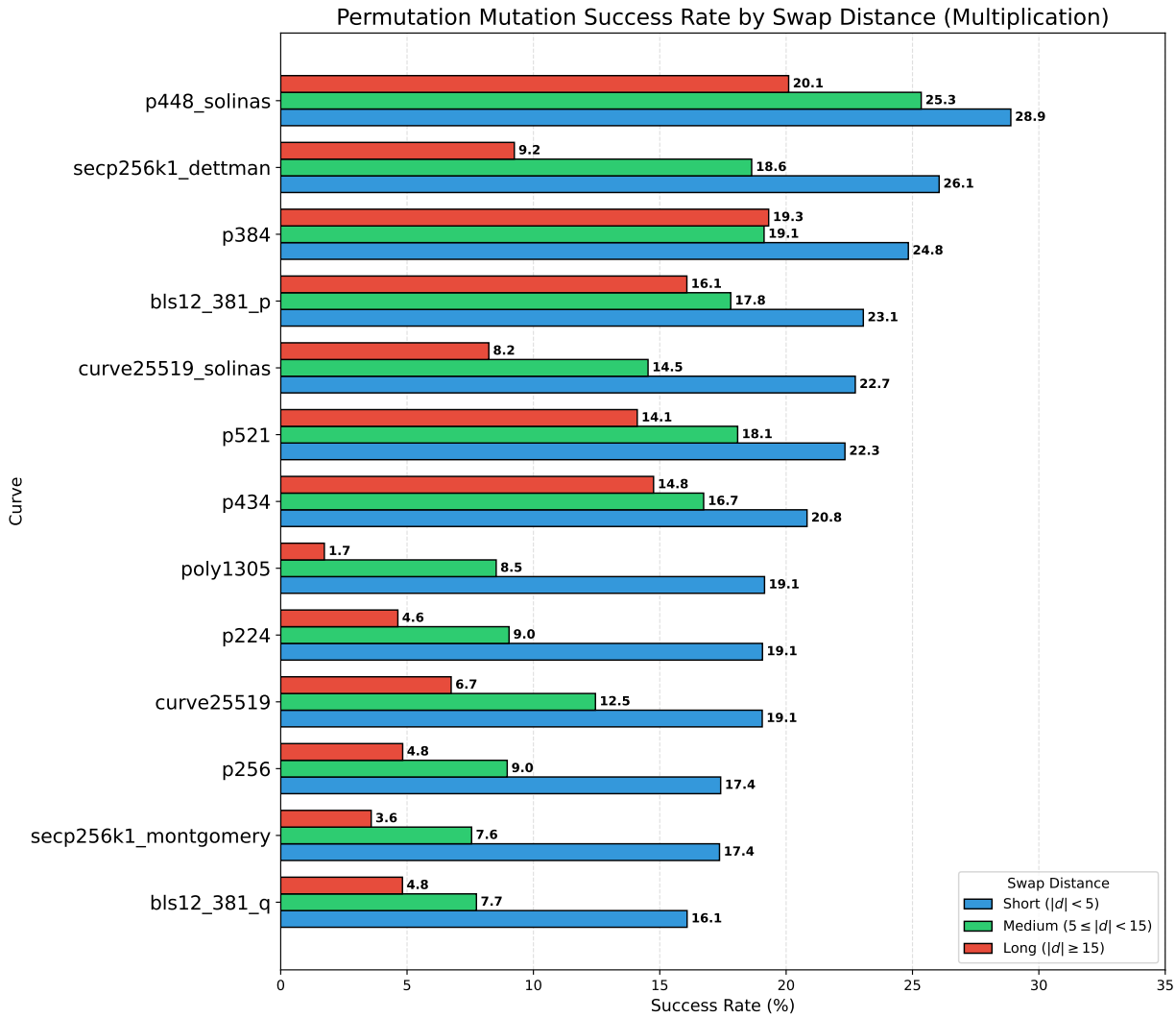

Figure 9: Permutation mutation success rates by swap distance for multiplication. Each bar shows the mean success rate across all 11 M_P:M_D ratio configurations. Short: $|d| < 5$, Medium: $5 \leq |d| < 15$, Long: $|d| \geq 15$.

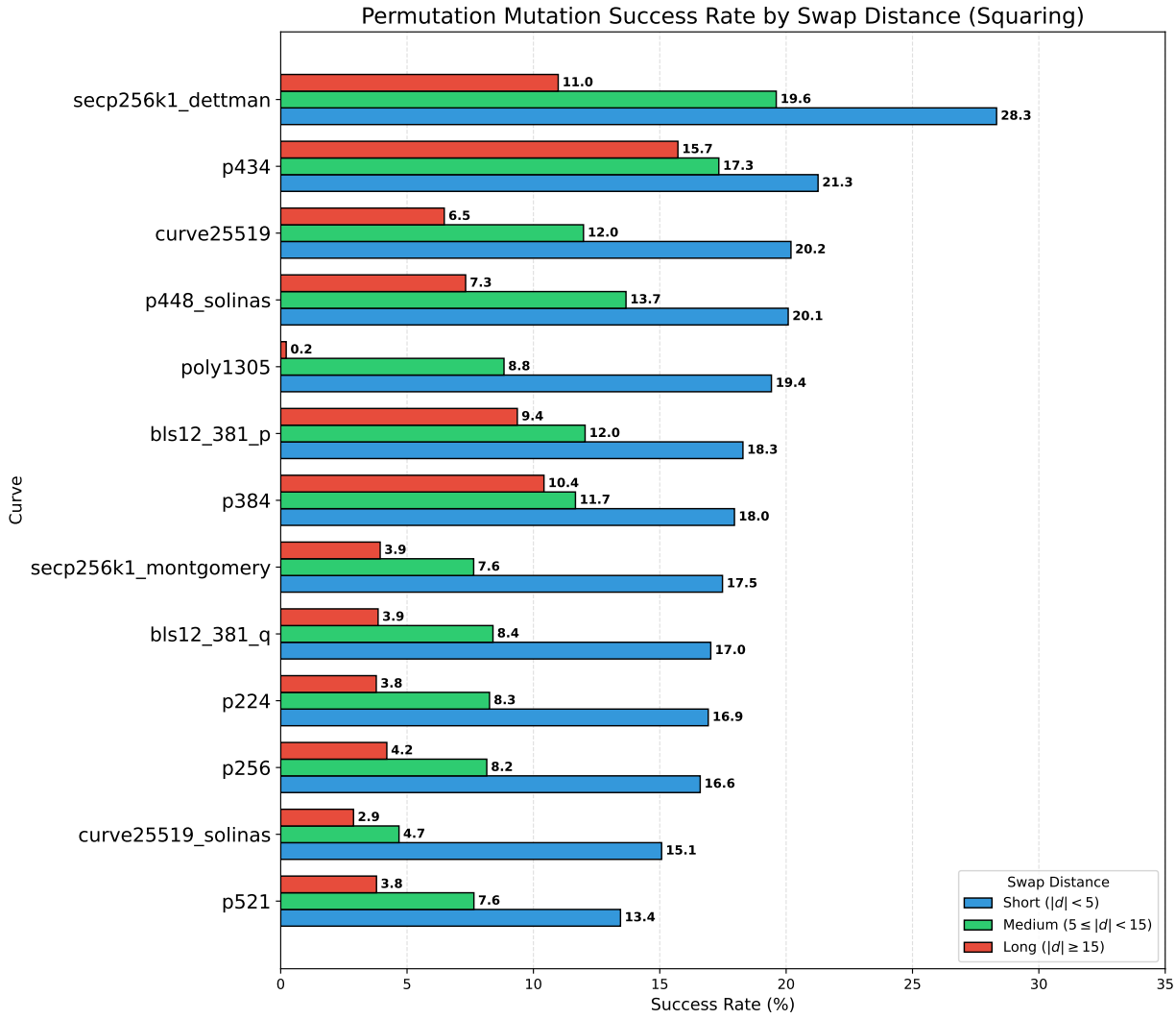

Figure 10: Permutation mutation success rates by swap distance for squaring. Each bar shows the mean success rate across all 11 M_P:M_D ratio configurations.

3.5 Summary of Findings

The experimental results yield several findings with implications for the design of internal optimizers.

First, despite Decision mutations exhibiting higher per-mutation success rates (25.9% vs. 18.8%), configurations dominated by Permutation mutations achieve superior final performance. The default M_P:M_D = 50:50 ratio yields a mean improvement of 20.6% across all experiments, whereas M_P:M_D = 90:10 achieves 24.9%—a relative gain of 21%. Ratios from 70:30 through 100:0 all exceed 22% mean improvement, while ratios below 30:70 fall below 20%. These results suggest that instruction scheduling contributes more to performance than instruction selection, and that the current default ratio could be adjusted to favor Permutation mutations.

Second, the success rate of short-distance swaps (20.0%) exceeds that of long-distance swaps (8.1%) by a factor of 2.5 on average. This pattern suggests that fine-grained, local instruction reordering is more productive than aggressive restructuring. However, the optimal swap distance distribution may be primitive-dependent: poly1305 shows a ratio of 19.8 between short and long success rates, while p384 shows only 1.4. An adaptive mutation strategy that adjusts distance preferences based on early optimization behavior could improve convergence.

Third, among Decision mutation types, argument reordering (AR) achieves a 28.5% success rate, while register spill placement (B&) achieves only 7.2%. This argument reordering being more successful than register spill placement suggests that computational effort could be redirected from low-yield categories to more productive ones. Reducing the frequency of B& mutations in favor of AR or FL mutations would likely improve optimization efficiency with the same computational cost.

4 LLVM Bridge

The primary goal of this work is to extend the capability of CryptOpt [20] to be able to optimize cryptographic code beyond those that are provided by Fiat Cryptography [21]. Since CryptOpt mainly uses inputs that are in the Fiat Intermediate Representation or Fiat IR format, the main task is to enable CryptOpt to accept inputs other than the Fiat IR. To achieve this goal, this work designs and implements a sequence of transformations whose endpoint is a format that is compatible with CryptOpt.

This work focuses on allowing CryptOpt to accept inputs in the LLVM intermediate representation or LLVM IR, as it is a popular and widely used format across different high-level languages. The outline of the framework is shown in Figure 11. A key challenge lies in bridging the semantic gap between the generic low-level LLVM IR and CryptOpt's domain-specific internal model for finite-field arithmetic.

We note that in the original CryptOpt work [20], it also includes a benchmark on a secp256k1 curve, which is used in Bitcoin Core and is not from Fiat Cryptography [21]. However, this is the only non-Fiat input that CryptOpt is able to process. This work's further investigation indicates that there are limitations in CryptOpt's ability to process more generic LLVM IR inputs.

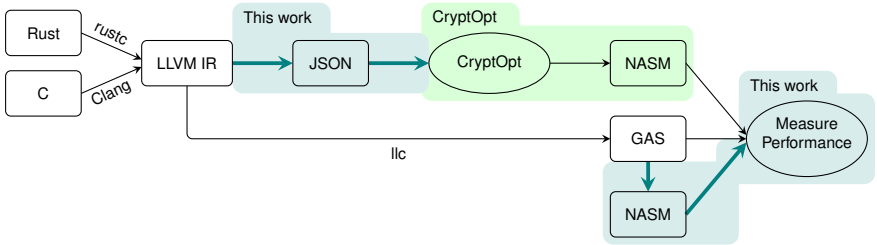

Figure 11: Overview of our framework.

4.1 Format Transformation

As this work targets accepting inputs in the LLVM IR format, cryptographic code implemented in high-level languages such as Rust or C must first be translated into the LLVM IR. This step can simply be achieved through using generic compilers, for example, *rustc* for Rust implementations or *Clang* for C implementations.

Once the LLVM IR input is obtained, the next and challenging step is to translate such low-level LLVM IR into the CryptOpt-acceptable format, which is referred to as a JSON format. To achieve this aim, this work starts by analyzing the LLVM IR and extracting specific components of each relevant LLVM IR operation using a regular expression. The extracted components are categorized into five parts: `name`, `operation`, `modifiers`, `datatype`, and `arguments`. The `name` stores the variable name, the `operation` stores the operation type, the `modifiers` store the optimization and semantic hints, the `datatype` stores the signed integer type, and the `arguments` store the operands.

To create the CryptOpt-acceptable JSON format, this work needs to ensure that operations captured in the JSON are expressible with CryptOpt's instructions. To this end, this work extends CryptOpt-supported instructions as we encounter cases where the existing CryptOpt instructions are insufficient and extension is necessary, for example, when using AUCurves-BLS12 [46] implementation as input. With this JSON format, it serves as a common intermediate representation that allows us to

partially connect to the existing Bitcoin Core Bridge of CryptOpt for the final format transformation to the CryptOpt-acceptable intermediate representation.

4.2 Bound Checks and Panic Blocks

It is noted that the LLVM IR from safe languages such as Rust may contain bound checks or panic blocks. A panic block is a compiler-inserted basic block that is executed when a runtime check fails (e.g., an out-of-bounds access) and typically transfers control to the language's panic/abort handler. While these are important for handling potential errors in the implementations, they do not directly impact the functionality of the code, assuming valid inputs and no errors. As such, these parts are not extracted or considered for further steps in our format transformation.

It is worth noting that the original CryptOpt [20], although its inputs from Fiat include bound checks at the source level, can still lose these bound checks in the LLVM IR, because compiler optimizations often remove definitions deemed unnecessary. In brief, the presence of these panic blocks in the IR is an artifact of the source language and compiler settings.

4.3 Variable Renaming and Address Reconciliation

Although LLVM IR is already in SSA form (i.e., each variable is defined exactly once), variable names may appear unorganized. Note that the original Bitcoin Core bridge benefits from sequential variable names (e.g., `x0`, `x1`, `x2`, and so on). On the other hand, the LLVM IR generated from Rust implementations does not necessarily follow this sequential naming convention. Instead, the variable names can be non-intuitive and irregular (e.g., `%0.i`, `%0.i14`, `%_x1`, `%_0.i2`, `%11`). Therefore, this work reorganizes the variable names and converts them into a sequential format, as this simplifies our workflow in the latter steps.

In addition to renaming variables, this work also handles the mismatch of the address convention between that of LLVM IR and CryptOpt. Note that the LLVM IR counts in bytes, whereas CryptOpt uses words. Hence, this work converts byte offsets to word indices to reconcile this discrepancy between LLVM IR's byte-based memory semantics and CryptOpt's word-based addressing.

4.4 Extend Supported Instructions

Recall that CryptOpt's main input format is Fiat IR. As such, CryptOpt does not natively handle LLVM IR operations. Even though the Bitcoin Core bridge of CryptOpt does support some LLVM operations, such support is rather limited. In particular, it only supports operations that are needed for converting the secp256k1 curve implemented in C into CryptOpt-acceptable JSON format.

In the format transformation of the LLVM IR into the JSON format for CryptOpt, two operations were encountered that are not supported by CryptOpt, namely, the native LLVM IR instructions SEXT (sign extension) and SELECT. To address this issue, these unsupported operations are transformed into CryptOpt instructions. The proposed solution involves mapping the two instructions to the CMOVZNZ (conditional move if zero/non-zero) instruction, which is supported by CryptOpt. The mapping process is described below.

4.4.1 Mapping SELECT to CMOVZNZ.

The SELECT instruction in LLVM IR chooses between two values based on a boolean condition without branching. CryptOpt's CMOVZNZ (conditional move if zero/non-zero) provides equivalent functionality but operates on integer flags rather than explicit booleans. An example of the SELECT instruction is shown below.

```
<result> = select i1 <cond>, i64 <val1>, i64 <val2>
```

It performs the following actions. If the condition `<cond>` is true (or has a value of 1), it assigns `<val1>` to `<result>`. Otherwise, it assigns `<val2>` to `<result>`. To map this into CryptOpt's instruction, we reinterpret `<cond>` as an integer flag (0 or 1). CryptOpt's CMOVZNZ instruction behaves similarly. It selects between two values based on whether a flag is zero or non-zero. The equivalent transformation in CryptOpt is:

```
<result> = cmovznz i1 <cond>, i64 <val2>, i64 <val1>
```

which does one of the two cases. If `<cond>` is true (non-zero), it selects `<val1>`. Otherwise, it selects `<val2>`. Essentially, the positions of `<val1>` and `<val2>` are swapped to match the semantics of the CMOVZNZ instruction, which chooses the second value if the condition is non-zero. It is noted that this transformation is valid because LLVM `i1` booleans can be safely treated as 1-bit integers (0 or 1). Also, CMOVZNZ makes the same binary decision based on whether the flag is zero or not (without using branches, similarly to SELECT). Therefore, the SELECT instruction can be safely transformed into CMOVZNZ without changing the program's logic.

4.4.2 Mapping SEXT to CMOVZNZ.

The SEXT in LLVM IR increases the bit width of a value while preserving its sign. For example, when extending a 1-bit boolean (`i1`) to a 64-bit integer (`i64`), SEXT does one of the following. If the input is true (or has a value of 1), the output is -1 (or `0xFFFFFFFFFFFFFFFF`). This is due to the sign extension of 1 in `i1` being equivalent to filling all the higher bits with 1s, i.e., the two's complement representation of -1 in `i64`. Otherwise, the output is 0 (or `0x0000000000000000`). An example of the SEXT instruction is shown below.

```
<result> = sext i1 <val3> to i64
```

It is interpreted as follows. The value `<val3>` is used as a condition. If it holds a value true (or 1), then the value -1 (or `0xFFFFFFFFFFFFFFFF`) is assigned to `<result>`. Otherwise, the value 0 (or `0x0000000000000000`) is assigned to `<result>`. It is observed that this functionality can be fulfilled by also using CryptOpt's CMOVZNZ instruction. In this context, the value `<val3>` represents the original 1-bit boolean value to be extended. It is used as the condition to select between two values depending on whether the bit is zero (false) or non-zero (true). That is, the sign extension can be performed in CryptOpt using:

```
<result> = cmovznz i1 <val3>, i64 0, i64 -1
```

which does one of the two cases. If `<val3>` is true (non-zero), it selects -1 (or `0xFFFFFFFFFFFFFFFF`). Otherwise, it selects 0 (or `0x0000000000000000`). This transformation is valid because the SEXT instruction is designed to extend the sign of a value, and the CMOVZNZ instruction can be used to select between two values based on a condition. Also, the two's complement representation of -1 in `i64` is `0xFFFFFFFFFFFFFFFF`, which matches the behavior of SEXT when extending a 1-bit boolean. Finally, the CMOVZNZ instruction does not introduce any branches, maintaining the same semantics as the SEXT operation.

5 Performance Evaluation of the LLVM Bridge

This section presents a comprehensive performance evaluation of both the original CryptOpt framework and the proposed LLVM Bridge. To ensure a fair and accurate assessment, experiments are conducted across different CPU core types (Performance-Cores vs. Efficiency-Cores) and distinct measurement methodologies are compared.

Recall from Chapter 2.4.3 that the original CryptOpt workflow relies on AssemblyLine [27] to assemble generated NASM code into an in-memory executable buffer for performance measurement. In contrast, the baseline for comparison is a shared object file (`.so`) produced by off-the-shelf compilers like *GCC* or *Clang* and loaded via the dynamic linker. These two approaches (running code from an in-memory buffer versus a loaded shared object) differ fundamentally in memory layout, loading mechanisms, and potential alignment biases, which can introduce performance artifacts independent of the code quality itself.

To address these discrepancies and isolate the true performance impact of the proposed optimizations, this work performs a multi-faceted evaluation:

1. Re-evaluation of the Original CryptOpt: The original Fiat bridge outputs are benchmarked on both P-Cores and E-Cores. Furthermore, the original AssemblyLine-based measurement is compared against a "non-AssemblyLine" approach (compiling NASM to a shared object by off-the-shelf compilers) to quantify the impact of the measurement harness itself.

2. Evaluation of the LLVM Bridge by our Comparison Framework: The new LLVM Bridge outputs are evaluated using our new, more rigorous assembly-to-assembly comparison methodology to ensure fairness.

3. Evaluation of the LLVM Bridge by MeasureSuite: The new LLVM Bridge outputs are also evaluated using CryptOpt's own comparison framework to demonstrate broad applicability. Additionally, benchmarking using the SUPERCOP [47] framework is performed to verify performance at the scalar multiplication level.

First, the re-evaluation of the original CryptOpt framework is presented. Then, the evaluation of the LLVM Bridge is presented in the following sections, utilizing both our own comparison framework and MeasureSuite.

5.1 Re-evaluation of Original CryptOpt

Before evaluating the LLVM Bridge outputs, this work first re-examines the performance characteristics of the original CryptOpt framework. Previous work [20] utilized AssemblyLine [27] to assemble generated code into in-memory executable buffers for measurement on P-Cores. In contrast, standard baselines are typically compiled into shared objects (`.so`) and loaded via the dynamic linker. This difference in loading mechanisms, memory alignment, and CPU core selection may introduce performance artifacts.

5.1.1 Re-evaluation Methodology.

To isolate these factors, we conducted the experiments without relying on AssemblyLine. The suite includes field arithmetic (multiplication and squaring) for 11 primitives, including Curve25519, P-256,

and secp256k1, which were originally measured and reported by CryptOpt [20]. The experiments are structured into three distinct configurations to decouple the impact of the CPU architecture from the measurement harness. Recall from Chapter 2.4 that CryptOpt employs a random local search algorithm, which means that, given the allocated computational budget, the optimizer may explore different starting points and directions many times. CryptOpt [20] used a budget of 200,000 mutation evaluations to keep the balance between execution time and exploration sufficient. In our experiments, however, we must compile the CryptOpt-generated NASM assembly every time to evaluate performance, which takes much longer than mapping the NASM assembly to an in-memory executable buffer using AssemblyLine on the fly. Hence, we reduce the computational budget to 100,000 evaluations to maintain a comparable total wall-clock time. All other hyperparameters, such as the bet ratio, remain consistent with the original study [20] to ensure fairness.

In addition, we measured performance on different CPU core types (P-Core and E-Core) in order to understand how the microarchitecture affects performance.

5.1.2 Re-evaluation Results.

Table 2 presents the speedup of CryptOpt-generated code over the baseline (Clang) for both Multiply and Square operations. There are three different parameters in the table: Core Type (P-Core or E-Core), AssemblyLine (Yes or No), and Evaluations (200k or 100k). Each value shows the speedup of the CryptOpt-generated code over the baseline (Clang) for both Multiply and Square operations. We highlight values greater than 1 in green to indicate that the CryptOpt-generated code is faster, and values less than 1 in orange to indicate it is slower.

Table 2: Re-evaluation of Original CryptOpt (Fiat Bridge). Speedup vs. Clang Baseline. Green values (>1) indicate CryptOpt outperforms the baseline; red values (<1) indicate slower performance.

	Multiply			**Square**		
Implementation	**Original [20]**	**E-Core**	**P-Core**	**Original [20]**	**E-Core**	**P-Core**
Core Type	*P-Core*	*E-Core*	*P-Core*	*P-Core*	*E-Core*	*P-Core*
AssemblyLine	*Yes*	*No*	*No*	*Yes*	*No*	*No*
Evaluations	*200k*	*100k*	*100k*	*200k*	*100k*	*100k*
Fiat Curve25519	1.25	1.03	1.29	1.22	0.96	1.24
Fiat Curve25519-Solinas	1.61	1.16	1.58	1.43	1.08	1.39
Fiat P-224	1.56	1.05	1.44	1.42	0.99	1.33
Fiat P-256	1.69	1.27	1.62	1.64	1.19	1.54
Fiat P-384	1.47	0.99	1.17	1.38	0.80	1.10
Fiat SIKE p434	1.91	1.00	1.48	1.81	0.93	1.35
Fiat Curve448	1.19	0.95	1.53	1.08	1.01	1.14
Fiat P-521	1.32	0.96	1.59	1.36	0.95	1.36
Fiat Poly1305	1.16	1.00	1.22	1.14	0.96	1.20
Fiat secp256k1-Dettman	1.08	0.95	1.18	1.05	0.97	1.11
Fiat secp256k1	1.85	1.07	1.45	1.76	1.12	1.31

We note that the original CryptOpt [20] results were obtained as the mean over ten different hard-

ware architectures, and even if we set the same parameters on the same hardware environment, we would not be able to reproduce the exact results due to the stochastic nature of the random local search algorithm and hardware variations. However, the results show that even without using AssemblyLine, the performance on P-Cores of the CryptOpt-generated code is still comparable to the original CryptOpt [20] results. That is, these results suggest AssemblyLine does not appear to introduce performance gains by the memory layout and mapping mechanism against the shared object file baseline. On the other hand, some large codebases, such as SIKE p434, cannot be explored sufficiently due to the reduced computational budget. We also note that the measured performance on E-Cores is lower in most cases than the baseline and the P-Core results. We attribute this to the fact that the CryptOpt output is specifically tuned for P-Cores, which can execute more instructions in parallel than E-Cores. Running on E-Cores therefore cannot fully exploit the assumed parallel execution capabilities, leading to lower performance than the off-the-shelf compiled code.

5.2 Evaluation of LLVM Bridge by our own comparison framework

5.2.1 Target Implementations

To demonstrate the generality of our framework, we benchmark 13 implementations comprising 25 routines (multiplication and squaring per implementation except for BLS12, which we benchmark only multiplication). In what follows, we detail the implementations used in our experiments for our benchmark.

Fiat Rust and Fiat C. As our first target group, we aim to directly use inputs from Rust and C implementations of Fiat Cryptography [21] (as opposed to Fiat IR) and optimize them. In particular, we extend the original Bitcoin Core of CryptOpt [20], which previously supported only a C-based secp256k1 curve [48], to further support Rust and C implementations from Fiat Cryptography, which we refer to as Fiat Rust and Fiat C, respectively. Note that both Fiat Rust and Fiat C are also formally verified by Fiat Cryptography. This implies that we inherit the proof of correctness for our optimized code by applying an equivalence checker against the inputs from CryptOpt and the Fiat Rust or Fiat C implementations.

AU Curves. For the second target group, we aim to support the Rust implementation of the BLS12 curve from the AUCurves project [46]. This project builds on the infrastructure of Fiat Cryptography [21] and bedrock2 [49] to provide a high-performance implementation of the BLS12-381, a pairing-friendly elliptic curve widely used in cryptographic applications. Supporting optimization for the AUCurves project has broad impacts on the community.

Dalek and RustCrypto. Our third target group is the implementations from curve25519-dalek [50] and RustCrypto/elliptic-curves [29]. The curve25519-dalek is a pure-Rust library that implements group operations on Ristretto [51] and Curve25519. It provides a clean and safe API for a wide range of elliptic-curve-based cryptographic protocols, including key agreement, signatures, anonymous credentials, range proofs, and zero-knowledge proofs. The secp256k1 implementation in RustCrypto/elliptic-curves is written in pure Rust, providing arbitrary group-based protocols on the secp256k1 curve.

OpenSSL. Our final target group is the implementations from OpenSSL [52], which is one of the most widely deployed cryptographic libraries, serving as the foundation for secure communications across the internet. It provides implementations of numerous cryptographic algorithms and protocols, and is used by countless applications worldwide. Given its critical role in real-world security infrastructures, OpenSSL implementations serve as an important benchmark.

5.2.2 Comparison Framework

Recall that our work is built on CryptOpt [20]. This implies that we could have reused CryptOpt's comparison framework. However, our analysis of CryptOpt's performance comparison indicates that there may be a subtle issue in how CryptOpt compares performance. We observe that CryptOpt does not directly compare assembly code with assembly code. Instead, CryptOpt reports the performance improvement of an in-memory executable buffer—produced at runtime by AssemblyLine [27] from the CryptOpt-generated Intel-syntax NASM assembly (`.asm`)—compared to a shared object file (`.so`). To be more precise, CryptOpt compiles Fiat C using *GCC* or *Clang* into a shared object file and uses it as a baseline when comparing with CryptOpt generated NASM assembly (see Figure 1). Such a comparison may introduce a bias and may not accurately reflect actual performance because in-memory executable buffers and shared objects differ in loading/relocation, layout, and alignment.

To remedy this issue, we develop a new performance comparison framework that promotes an apples-to-apples comparison. Our framework is depicted in Figure 11. After the input code is compiled into the LLVM IR (using off-the-shelf compilers), it goes through either our optimization pipeline (which invokes CryptOpt to output assembly) or an off-the-shelf assembler (*llc*). For the former, the NASM assembly output by CryptOpt is further assembled into an object file via NASM `elf64`. For the latter, we obtain GAS assembly from *llc* with the highest optimization level (`-O3`) and with BMI2/ADX instructions enabled, which is further assembled into an object file via *Clang*. Additionally, we build a translator to convert the GAS assembly into NASM assembly, which is also further assembled into an object file via NASM `elf64`. Finally, we compare the execution time of the following three object files:

1. A CryptOpt-generated NASM assembly, assembled via NASM `elf64`
2. An off-the-shelf compiler-generated GAS assembly, assembled via *Clang*
3. A translated NASM assembly (from GAS), assembled via NASM `elf64`

We link all three object files against the same set of static libraries and the same measurement harness (identical link flags and runtime configurations) to measure their performance under identical conditions. Through this approach, we ensure that object files are compared with other object files, enabling more accurate performance analysis. We note that comparing the CryptOpt-generated NASM assembly with the off-the-shelf compiler-generated GAS assembly is sufficient to demonstrate the performance improvement. We add the comparison to the translated NASM assembly (from GAS) to highlight potential differences between the two assembly formats.

5.2.3 Experimental Setup

We perform our experiments on a 13th Gen Intel Core i7-1360P (Raptor Lake-P) with 8 cores and 16 threads, running Ubuntu 22.04.4 LTS. We use the RDTSC instruction with memory barriers (MFENCE)

to measure the execution time. In what follows, we describe approaches that we employ to reduce environmental factors that could affect our measurement results. Also, we measured the performance in this case on the E-Core and P-Core.

Noise Reduction. Prior to each experiment, we pin the benchmark target function to a single physical core and dedicate another core for non-benchmarking activities, such as the OS and background jobs. We disable the Simultaneous Multi-Threading (SMT) to prevent logical sibling cores from competing for resources. We further disable the Turbo Boost and dynamic frequency scaling and fix the frequency at the lowest non-turbo P-state to keep the clock stable. We also turn off the Address Space Layout Randomization (ASLR) to ensure that code and data reside at identical addresses from one repetition to the next. Finally, we stop common daemons, such as *cron*, *irqbalance*, and *systemd-timesyncd*, and redirect noisy interrupt lines away from the benchmark core to reduce potential interference with our measurements.

5.2.4 Performance Results

This subsection reports our evaluation results, where we consider multiple aspects. We first measure the performance of *field arithmetic* operations, namely, field multiplication and field squaring, which are important arithmetic building blocks. Next, we integrate our evaluation with the SUPERCOP framework and present the performance of *scalar multiplication*, a core operation in elliptic-curve cryptography. Finally, we revisit the side-channel security of our produced code and test its constant-time properties.

We use Clang with the `-O3` optimization flag as our baseline, which represents the highest optimization level available in the LLVM/Clang compiler toolchain. This level enables all standard optimizations including aggressive inlining, loop unrolling, and vectorization, making it a strong baseline for performance comparison.

Ours vs Off-the-Shelf Compilers. We present the performance comparison of our LLVM bridge CryptOpt-optimized x86-64 assembly code against each of the two compiler-generated assembly code in Table 3 (E-Core), and Table 7 (P-Core). For each implementation, we evaluate both field multiplication (Multiply) and field squaring (Square) operations. We report the performance as a ratio (our code / baseline), highlighting ratios greater than 1 in green to indicate that our code is faster and ratios less than 1 in orange to indicate slowdowns. More detailed results (e.g., cycle counts per function call, confidence intervals) are available in Table 4 and Table 5 (E-Core), Table 8, and Table 9 (P-Core) for Rust and C implementations, respectively.

These results indicate a mixed outcome. In the E-Core case, our CryptOpt-optimized code via the LLVM bridge sometimes outperforms off-the-shelf compiler-generated code, but is often slower overall. We observe notable wins for Curve25519-dalek squaring (1.46 against NASM). However, in a large majority of cases we observe slowdowns, particularly for AUCurves-BLS12 and OpenSSL Curve25519, where our generated code is substantially slower than both baselines.

On the other hand, in the P-Core case, our CryptOpt-optimized code via the LLVM bridge demonstrates better performance, with the majority of implementations showing speedups. We observe notable improvements across most implementations, with the highest gains for Fiat-NIST-P448 multiplication (1.38 and 1.39 for GAS and NASM, respectively) and even Curve25519 (OpenSSL vs Hand-

Table 3: Assembly-to-assembly performance comparison on E-Core CPUs. The GAS column compares CryptOpt-generated NASM against Clang -O3-generated GAS. The NASM column compares CryptOpt-generated NASM against Clang -O3-generated GAS converted to NASM. Green values (>1) indicate CryptOpt outperforms the baseline; red values (<1) indicate slower performance.

		Multiply		Square	
Implementation	**Lang**	**GAS**	**NASM**	**GAS**	**NASM**
Curve25519-dalek [50]	Rust	0.87	0.89	1.44	1.46
EC-Secp256k1 [29]	Rust	0.83	0.83	1.20	1.20
Fiat-Poly1305 [53]	Rust	0.87	0.89	1.01	1.01
Fiat-Curve25519 [53]	Rust	0.92	0.92	0.93	0.93
Fiat-NIST-P448 [53]	Rust	0.85	0.84	0.92	0.91
Fiat-Secp256k1 [53]	Rust	0.82	0.80	0.86	0.82
AUCurves-BLS12 [46]	Rust	0.59	0.59	—	—
Fiat-NIST-P448 [54]	C	0.94	0.97	1.08	1.04
Fiat-Curve25519 [54]	C	1.01	1.01	0.99	0.99
Fiat-Poly1305 [54]	C	0.82	0.83	0.99	0.99
Fiat-Secp256k1 [54]	C	0.84	0.86	0.85	0.83
OpenSSL Implementations					
Curve25519 [52]	C (vs OTS)	0.94	0.92	0.97	0.97
Curve25519 [52]	C (vs Hand-tuned)	0.60	0.63	0.47	0.48
P448 [52]	C	0.79	0.80	0.89	0.88

Note: '—' indicates that the implementation is not supported by our workflow.

tuned) multiplication (1.26 for NASM). Most Rust and C implementations show consistent speedups ranging from 1.04 to 1.39. However, we still observe slowdowns for AUCurves-BLS12 (0.61 and 0.59 for multiply) and marginal slowdowns for some squaring operations, particularly Fiat-Poly1305 (Rust) at 0.94.

This performance gap between the E-Core and P-Core architectures indicates that CryptOpt focuses on generating code that is faster than the off-the-shelf compiler-generated code specifically for the P-Core architecture, but not for the E-Core architecture.

Constant-timeness. To empirically verify that our CryptOpt LLVM-Bridge produces code that preserves constant-time properties, we use a testing framework dudect [42], which detects variations in the execution time due to secret-dependent operations. We embed dudect directly into our compilation flow; every CryptOpt assembly file is wrapped in a C harness generated on the fly. The harness first prepares two input sets: one is completely zeroed and the other is filled with random data, yet respecting the specification (e.g., field's modulus limits). It then calls the test targets repeatedly while timing each invocation with the high-resolution RDTSC counter. After gathering 100,000 cycle counts for both input sets, dudect applies Welch [43]'s t-test. Upon completion, it reports either PASS (i.e., no evidence of data-dependent timing) or FAIL (i.e., further investigation is necessary). We test every

Table 4: Detailed statistics for Rust implementations (E-Core)

Implementation	Operation	ASM	Median	Mean	(95% CI)	σ
Curve25519-dalek [50]	Multiply	**Ours**	**158**	**158.00**	**(158.00 , 158.00)**	**0.00**
		NASM	140	140.00	(140.00 , 140.00)	0.00
		GAS	138	138.25	(138.07 , 138.43)	0.43
	Square	NASM	229	226.81	(225.39 , 228.23)	3.87
		GAS	225	225.46	(225.26 , 225.66)	0.50
		Ours	**157**	**158.19**	**(157.64 , 158.75)**	**1.51**
EC-secp256k1 [29]	Multiply	**Ours**	**303**	**301.65**	**(300.07 , 303.22)**	**4.30**
		NASM	252	252.65	(251.09 , 254.20)	4.24
		GAS	251	253.97	(252.68 , 255.26)	3.51
	Square	NASM	328	328.62	(327.41 , 329.82)	3.01
		GAS	327	327.00	(327.00 , 327.00)	0.00
		Ours	**273**	**272.81**	**272.37 , 273.25)**	**1.20**
Fiat-Poly1305 [53]	Multiply	**Ours**	**93**	**93.00**	**(93.00 , 93.00)**	**0.00**
		NASM	83	83.00	(83.00 , 83.00)	0.00
		GAS	81	81.00	(81.00 , 81.00)	0.00
	Square	NASM	84	84.00	(84.00 , 84.00)	0.00
		GAS	84	84.00	(84.00 , 84.00)	0.00
		Ours	**83**	**83.00**	**(83.00 , 83.00)**	**0.00**
Fiat-Curve25519 [53]	Multiply	**Ours**	**173**	**173.41**	**(173.14 , 173.67)**	**0.68**
		NASM	159	159.00	(159.00 , 159.00)	0.00
		GAS	159	158.55	(158.37 , 158.73)	0.50
	Square	**Ours**	**152**	**152.00**	**(152.00 , 152.00)**	**0.00**
		NASM	142	141.68	(141.51 , 141.85)	0.47
		GAS	142	141.52	(141.33 , 141.70)	0.50
Fiat-P448 [53]	Multiply	**Ours**	**436**	**435.48**	**(435.27 , 435.70)**	**0.56**
		GAS	371	370.81	(370.04 , 371.57)	2.09
		NASM	364	362.76	(361.69 , 363.82)	2.81
	Square	**Ours**	**322**	**322.08**	**(321.97 , 322.19)**	**0.27**
		GAS	297	294.48	(292.92 , 296.05)	4.26
		NASM	294	294.10	(293.05 , 295.14)	2.86
Fiat-secp256k1 [53]	Multiply	**Ours**	**240**	**240.00**	**(240.00 , 240.00)**	**0.00**
		GAS	197	196.71	(196.28 , 197.14)	1.17
		NASM	192	193.61	(192.76 , 194.47)	2.34
	Square	**Ours**	**222**	**221.61**	**(221.42 , 221.80)**	**0.49**
		GAS	192	188.61	(186.88 , 190.35)	4.73
		NASM	182	182.00	(182.00 , 182.00)	0.00
BLS12 [46]	Multiply	**Ours**	**835**	**834.65**	**(833.69 , 835.60)**	**2.61**
		NASM	493	493.40	(493.13 , 493.67)	0.71
		GAS	492	492.39	(492.11 , 492.66)	0.75

Note: '(95% CI)' indicates the lower bound and the upper bound of the 95% confidence interval. The symbol 'σ' denotes the standard deviation.

Table 5: Detailed statistics for Fiat-Crypto C implementations (E-Core)

Implementation	Operation	ASM	Median	Mean	(95% CI)			σ
Fiat-P448 [54]	Multiply	**Ours**	**419**	**419.10**	**(418.49**	**,**	**419.72)**	**1.63**
		NASM	405	407.90	(406.02	,	409.78)	5.13
		GAS	395	397.94	(395.95	,	399.92)	5.42
	Square	GAS	322	322.77	(322.20	,	323.34)	1.42
		NASM	314	313.81	(311.31	,	316.31)	6.82
		Ours	**301**	**301.43**	**(301.20**	**,**	**301.65)**	**0.49**
Fiat-Curve25519 [54]	Multiply	GAS	158	158.00	(158.00	,	158.00)	0.00
		NASM	158	157.65	(157.47	,	157.82)	0.48
		Ours	**157**	**157.00**	**(157.00**	**,**	**157.00)**	**0.00**
	Square	**Ours**	**143**	**143.00**	**(143.00**	**,**	**143.00)**	**0.00**
		NASM	141	141.00	(141.00	,	141.00)	0.00
		GAS	141	140.19	(139.42	,	140.96)	2.10
Fiat-Poly1305 [54]	Multiply	**Ours**	**99**	**99.00**	**(99.00**	**,**	**99.00)**	**0.00**
		NASM	82	82.00	(82.00	,	82.00)	0.00
		GAS	81	81.00	(81.00	,	81.00)	0.00
	Square	**Ours**	**85**	**85.00**	**(85.00**	**,**	**85.00)**	**0.00**
		NASM	84	84.00	(84.00	,	84.00)	0.00
		GAS	84	84.00	(84.00	,	84.00)	0.00
Fiat-secp256k1 [54]	Multiply	**Ours**	**232**	**232.00**	**(232.00**	**,**	**232.00)**	**0.00**
		NASM	199	198.76	(198.57	,	198.95)	0.50
		GAS	196	196.14	(195.92	,	196.35)	0.57
	Square	**Ours**	**220**	**220.41**	**(220.21**	**,**	**220.60)**	**0.49**
		GAS	188	188.00	(188.00	,	188.00)	0.00
		NASM	182	182.20	(181.71	,	182.69)	1.30

Note: '(95% CI)' indicates the lower bound and the upper bound of the 95% confidence interval. The symbol 'σ' denotes the standard deviation.

Table 6: Detailed statistics for OpenSSL C implementations (E-Core)

Implementation	Operation	ASM	Median	Mean	(95% CI)			σ
Curve25519 [52]	Multiply	**Ours**	**167**	**167.00**	**(167.00**	**,**	**167.00)**	**0.00**
		GAS	157	155.48	(154.65	,	156.32)	2.27
		NASM	153	153.00	(153.00	,	153.00)	0.00
		Hand NASM	105	105.00	(105.00	,	105.00)	0.00
		Hand GAS	101	100.58	(100.38	,	100.78)	0.55
	Square	**Ours**	**148**	**148.00**	**(148.00**	**,**	**148.00)**	**0.00**
		GAS	143	141.65	(140.86	,	142.43)	2.13
		NASM	143	141.55	(140.67	,	142.43)	2.41
		Hand NASM	71	71.00	(71.00	,	71.00)	0.00
		Hand GAS	70	70.26	(70.10	,	70.42)	0.44
P448 [52]	Multiply	**Ours**	**403**	**402.69**	**(402.51**	**,**	**402.88)**	**0.46**
		GAS	320	321.39	(320.69	,	322.09)	1.91
		NASM	321	322.19	(321.18	,	323.20)	2.75
	Square	**Ours**	**308**	**308.19**	**(308.00**	**,**	**308.39)**	**0.48**
		GAS	273	273.46	(273.26	,	273.66)	0.50
		NASM	271	274.06	(272.05	,	276.08)	5.49

Note: '(95% CI)' indicates the lower bound and the upper bound of the 95% confidence interval. The symbol 'σ' denotes the standard deviation. 'Hand' refers to the hand-optimized assembly implementations included in OpenSSL.

function listed in Table 3. Except for the AUCurves-BLS12 [46], which crashes with a segmentation fault during the test due to the large codebase, all other functions return `PASS`, providing empirical evidence consistent with the constant-time behavior of our produced code under the tested conditions.

5.3 Evaluation of LLVM Bridge by MeasureSuite

Recall from Chapter 5.1 that our re-evaluation indicates AssemblyLine does not appear to yield a performance gain due to memory layout and mapping mechanisms compared to the shared object file baseline. Thus, we evaluate our LLVM Bridge outputs using CryptOpt's [20] own comparison framework.

5.3.1 Experimental Setup

We have different experimental setup phases for Optimization Phase and Evaluation Phase. The Optimization Phase focuses on generating higher-performance assembly code and does not necessarily measure the performance in a strict hardware environment. The Evaluation Phase, on the other hand, focuses on measuring the performance in a strict hardware environment.

Optimization Phase For the experimental setup, we follow the original CryptOpt [20] configuration. We pin the optimization processes to P-Cores using `taskset` to reduce noise due to context switching. We run up to three optimizations for different primitives in parallel, keeping remaining P-Cores free for general OS activity. Each optimization process consists of 200,000 mutation eval-

Table 7: Assembly-to-assembly performance comparison on P-Core CPUs. The GAS column compares CryptOpt-generated NASM against Clang -O3-generated GAS. The NASM column compares CryptOpt-generated NASM against Clang -O3-generated GAS converted to NASM. Green values (>1) indicate CryptOpt outperforms the baseline; red values (<1) indicate slower performance.

		Multiply		Square	
Implementation	**Lang**	**GAS**	**NASM**	**GAS**	**NASM**
Curve25519-dalek [50]	Rust	1.11	1.11	1.08	1.05
EC-Secp256k1 [29]	Rust	1.05	1.06	1.02	1.02
Fiat-Poly1305 [53]	Rust	1.10	1.10	0.94	0.94
Fiat-Curve25519 [53]	Rust	1.13	1.13	1.16	1.16
Fiat-NIST-P448 [53]	Rust	1.38	1.39	1.02	0.99
Fiat-Secp256k1 [53]	Rust	1.04	1.04	1.07	1.07
AUCurves-BLS12 [46]	Rust	0.61	0.59	—	—
Fiat-NIST-P448 [54]	C	1.39	1.39	0.99	0.99
Fiat-Curve25519 [54]	C	1.06	1.06	1.12	1.12
Fiat-Poly1305 [54]	C	1.05	1.05	1.06	1.06
Fiat-Secp256k1 [54]	C	1.05	1.05	1.09	1.09
	OpenSSL Implementations				
Curve25519 [52]	C (vs OTS)	1.09	1.09	1.13	1.13
Curve25519 [52]	C (vs Hand-tuned)	1.16	1.26	1.10	1.13
P448 [52]	C	1.24	1.23	1.00	1.00

Note: '—' indicates that the implementation is not supported by our workflow.

Table 8: Detailed statistics for Rust implementations (P-Core)

Implementation	Operation	ASM	Median	Mean	(95% CI)	σ
Curve25519-dalek [50]	Multiply	**Ours**	**44**	**44.00**	**(44.00 , 44.00)**	**0.00**
		NASM	49	49.00	(49.00 , 49.00)	0.00
		GAS	49	49.04	(48.90 , 49.19)	0.35
	Square	NASM	41	41.45	(41.27 , 41.63)	0.50
		GAS	42	41.68	(41.51 , 41.85)	0.47
		Ours	**39**	**39.00**	**(39.00 , 39.00)**	**0.00**
EC-secp256k1 [29]	Multiply	**Ours**	**62**	**62.00**	**(62.00 , 62.00)**	**0.00**
		NASM	66	65.54	(65.34 , 65.73)	0.50
		GAS	65	65.29	(65.12 , 65.46)	0.45
	Square	NASM	57	57.00	(57.00 , 57.00)	0.00
		GAS	57	57.00	(57.00 , 57.00)	0.00
		Ours	**56**	**55.55**	**(55.37 , 55.73)**	**0.50**
Fiat-Poly1305 [53]	Multiply	**Ours**	**20**	**20.48**	**(20.30 , 20.67)**	**0.50**
		NASM	22	22.00	(22.00 , 22.00)	0.00
		GAS	22	22.00	(22.00 , 22.00)	0.00
	Square	NASM	15	15.00	(15.00 , 15.00)	0.00
		GAS	15	15.00	(15.00 , 15.00)	0.00
		Ours	**16**	**16.00**	**(16.00 , 16.00)**	**0.00**
Fiat-Curve25519 [53]	Multiply	**Ours**	**45**	**45.00**	**(45.00 , 45.00)**	**0.00**
		NASM	51	51.00	(51.00 , 51.00)	0.00
		GAS	51	50.71	(50.52 , 50.90)	0.52
	Square	**Ours**	**32**	**32.00**	**(32.00 , 32.00)**	**0.00**
		NASM	37	37.00	(37.00 , 37.00)	0.00
		GAS	37	37.00	(37.00 , 37.00)	0.00
Fiat-P448 [53]	Multiply	**Ours**	**149**	**149.36**	**(149.06 , 149.65)**	**0.77**
		GAS	206	206.45	(206.20 , 206.71)	0.58
		NASM	207	206.72	(206.40 , 207.04)	0.78
	Square	**Ours**	**91**	**90.58**	**(90.40 , 90.76)**	**0.49**
		GAS	93	92.52	(92.29 , 92.74)	0.57
		NASM	90	90.42	(90.24 , 90.60)	0.49
Fiat-secp256k1 [53]	Multiply	**Ours**	**57**	**57.00**	**(57.00 , 57.00)**	**0.00**
		GAS	59	59.39	(59.19 , 59.59)	0.55
		NASM	59	59.39	(59.19 , 59.59)	0.55
	Square	**Ours**	**44**	**44.00**	**(44.00 , 44.00)**	**0.00**
		GAS	47	47.00	(47.00 , 47.00)	0.00
		NASM	47	47.00	(47.00 , 47.00)	0.00
BLS12 [46]	Multiply	**Ours**	**511**	**511.30**	**(510.67 , 511.93)**	**1.70**
		NASM	301	301.17	(299.93 , 302.41)	3.33
		GAS	311	310.97	(309.80 , 312.14)	3.20

Note: '(95% CI)' indicates the lower bound and the upper bound of the 95% confidence interval. The symbol 'σ' denotes the standard deviation.

Table 9: Detailed statistics for Fiat-Crypto C implementations (P-Core)

Implementation	Operation	ASM	Median	Mean	(95% CI)			σ
Fiat-P448 [54]	Multiply	**Ours**	**151**	**150.62**	**(150.32**	**,**	**150.93)**	**0.81**
		NASM	210	209.71	(209.41	,	210.02)	0.80
		GAS	210	209.58	(209.32	,	209.84)	0.71
	Square	GAS	87	87.00	(87.00	,	87.00)	0.00
		NASM	87	87.00	(87.00	,	87.00)	0.00
		Ours	**88**	**88.45**	**(88.21**	**,**	**88.70)**	**0.66**
Fiat-Curve25519 [54]	Multiply	GAS	51	51.00	(51.00	,	51.00)	0.00
		NASM	51	51.00	(51.00	,	51.00)	0.00
		Ours	**48**	**47.48**	**(47.28**	**,**	**47.69)**	**0.56**
	Square	**Ours**	**33**	**33.00**	**(33.00**	**,**	**33.00)**	**0.00**
		NASM	37	37.00	(37.00	,	37.00)	0.00
		GAS	37	37.00	(37.00	,	37.00)	0.00
Fiat-Poly1305 [54]	Multiply	**Ours**	**21**	**20.71**	**(20.54**	**,**	**20.88)**	**0.45**
		NASM	22	22.00	(22.00	,	22.00)	0.00
		GAS	22	22.00	(22.00	,	22.00)	0.00
	Square	**Ours**	**16**	**16.00**	**(16.00**	**,**	**16.00)**	**0.00**
		NASM	17	17.00	(17.00	,	17.00)	0.00
		GAS	17	17.00	(17.00	,	17.00)	0.00
Fiat-secp256k1 [54]	Multiply	**Ours**	**56**	**55.68**	**(55.51**	**,**	**55.85)**	**0.47**
		NASM	59	59.00	(59.00	,	59.00)	0.00
		GAS	59	59.00	(59.00	,	59.00)	0.00
	Square	**Ours**	**43**	**43.00**	**(43.00**	**,**	**43.00)**	**0.00**
		GAS	47	47.00	(47.00	,	47.00)	0.00
		NASM	47	47.00	(47.00	,	47.00)	0.00

Note: '(95% CI)' indicates the lower bound and the upper bound of the 95% confidence interval. The symbol 'σ' denotes the standard deviation.

Table 10: Detailed statistics for OpenSSL C implementations (P-Core)

Implementation	Operation	ASM	Median	Mean	(95% CI)			σ
Curve25519 [52]	Multiply	**Ours**	**43**	**43.00**	**(43.00**	**,**	**43.00)**	**0.00**
		GAS	47	47.00	(47.00	,	47.00)	0.00
		NASM	47	47.00	(47.00	,	47.00)	0.00
		Hand NASM	54	53.65	(53.47	,	53.82)	0.48
		Hand GAS	50	50.39	(50.21	,	50.57)	0.49
	Square	**Ours**	**30**	**30.00**	**(30.00**	**,**	**30.00)**	**0.00**
		GAS	34	34.00	(34.00	,	34.00)	0.00
		NASM	34	34.00	(34.00	,	34.00)	0.00
		Hand NASM	34	34.00	(34.00	,	34.00)	0.00
		Hand GAS	33	33.32	(33.07	,	33.58)	0.66
P448 [52]	Multiply	**Ours**	**91**	**91.26**	**(91.10**	**,**	**91.42)**	**0.44**
		GAS	113	112.55	(112.26	,	112.84)	0.77
		NASM	112	112.24	(111.90	,	112.58)	0.90
	Square	**Ours**	**68**	**68.45**	**(68.27**	**,**	**68.63)**	**0.50**
		GAS	68	67.60	(67.37	,	67.83)	0.61
		NASM	68	67.45	(67.21	,	67.70)	0.66

Note: '(95% CI)' indicates the lower bound and the upper bound of the 95% confidence interval. The symbol 'σ' denotes the standard deviation. 'Hand' refers to the hand-optimized assembly implementations included in OpenSSL.

uations; we use 20,000 evaluations for the Bet phase and the remaining 180,000 for the exploration phase.

Evaluation Phase Once the optimization is complete, we evaluate the performance. To reduce system noise, we fix the CPU frequency, disable Turbo Boost, and set the CPU governor to performance mode. We note that we apply these settings only when evaluating performance, not during optimization, since the optimization process itself takes a considerable amount of time depending on the codebase. It would be impractical to maintain such low-noise conditions during the entire optimization process and also the noise reduction itself would not affect the optimization process itself.

For cycle measurement, we use a fixed batch size of 400 function invocations per batch, with 31 batches per measurement, following the original CryptOpt methodology. To ensure statistical reliability, we collect 10 measurement samples and validate them using a two-sample t-test with a 99.5% confidence interval. If the samples are statistically apparently different, the measurement is repeated until a consistent result is obtained.

5.3.2 Results

5.3.3 Comparison against Off-the-Shelf Compilers by MeasureSuite.

This subsection presents the results obtained using the MeasureSuite comparison framework. We compare the performance against the baseline (Clang for C and *rustc* for Rust) for both Multiply and Square operations. We report the speedup, highlighting improvements in green to indicate that our code is faster and slowdowns in orange. For instance, for the Fiat-NIST-P448 Rust implementation,

Table 11: Performance comparison between our LLVM bridge CryptOpt-optimized code vs compiler-generated code via MeasureSuite. Green values (>1) indicate CryptOpt outperforms the baseline; red values (<1) indicate slower performance.

Implementation	Lang	Multiply	Square
Fiat-NIST-P448 [53]	Rust	1.56	0.94
Curve25519-dalek [50]	Rust	1.28	0.93
Fiat-Curve25519 [53]	Rust	1.18	1.02
Fiat-Poly1305 [53]	Rust	1.08	1.01
Fiat-Secp256k1 [53]	Rust	1.08	0.96
EC-Secp256k1 [29]	Rust	1.01	0.90
AUCurves-BLS12 [46]	Rust	0.57	—
Fiat-NIST-P448 [54]	C	1.60	1.07
Fiat-Curve25519 [54]	C	1.21	1.02
Fiat-Poly1305 [54]	C	1.10	1.03
Fiat-Secp256k1 [54]	C	1.06	0.98
OpenSSL Implementations			
Curve25519 [52]	C	1.16	1.04
P448 [52]	C	0.98	1.09

Note: '—' indicates that the implementation is not supported by our workflow.

the speedup is 1.56, which means our code is 1.56 times faster than the baseline generated by *rustc* for multiplication. Furthermore, even against the OpenSSL Curve25519 implementation, our code is 1.16 times faster for multiplication.

Scalar Multiplication and SUPERCOP. To demonstrate the effectiveness of our work beyond the level of basic arithmetic operations, we perform benchmarking on *scalar multiplication*. To this end, we use the SUPERCOP framework [47] in our evaluation.

To benchmark, we integrate our CryptOpt-generated assembly into existing SUPERCOP framework [47]. For Curve25519, we replace the field arithmetic operations in the OpenSSL's fe51 implementation with assembly generated from multiple sources: Fiat Rust [53], Fiat C [54], Dalek [50], and OpenSSL [52]. These integrations produce O'SSL-fe51 + Fiat Rust + ours, O'SSL-fe51 + Fiat C + ours, O'SSL-fe51 + Dalek + ours, and O'SSL-fe51 + OpenSSL-C-OTS + our, respectively. For secp256k1, we replace field arithmetic operations in libsecp256k1 with assembly generated from Fiat Rust [53], Fiat C [54], and RustCrypto [29], constructing libsecp256k1 + Fiat Rust + ours, libsecp256k1 + Fiat C + ours, and libsecp256k1 + RustCrypto + ours, respectively. Additionally, we evaluate several other implementations that are part of the standard SUPERCOP suite.

Note that for each implementation, SUPERCOP systematically explores multiple combinations of compilers (*GCC* and *Clang*) and compiler optimization levels (`-O`, `-O2`, `-O3`, `-Os`) with various architecture-specific flags (`-march=native`, `-mtune=native`, `-mcpu=native`). SUPERCOP reports the execution time for two base-point multiplications and two variable-point multiplica-

tions using the fastest compiler configuration. We evaluate each implementation on a single hardware platform, the same compiler configuration mentioned in Chapter 5.3.1, and report cycle counts rounded to the nearest 1000 cycles.

We present the benchmarking results for scalar multiplication in Table 12. These results indicate that our approach produces code with competitive performance with the others. For example, Curve25519 of O'SSL-fe51 + OpenSSL-C-OTS + ours matches the cycle counts of O'SSL-fe51 + Hand-tuned [52] and O'SSL-fe51 + OpenSSL-C-OTS [52].

Table 12: Scalar multiplication measured with SUPERCOP (Best Cycles)

Implementation	**Language**	k **Cycles**
Curve25519		
Reference Implementation [55]	C	13423
Reference Implementation (ref10) [56]	C	1318
Donna [57]	asm-v	1040
Rust-Crypto (Dalek) [50]	Rust	673
Sandy2x [5]	asm-v	581
O'SSL-fe51 + Dalek [50] + **(ours)**	asm	**543**
AMD64-64 [15]	asm	542
O'SSL-fe51 + Fiat C [54] + **(ours)**	asm	**540**
O'SSL-fe51 + Fiat Rust [53] + **(ours)**	asm	**540**
Donna-C64 [57]	C	536
O'SSL-fe51 + Fiat C [54]	C	520
AMD64-51 [15]	asm	505
O'SSL-fe51 + OpenSSL-C-OTS [52]	C	501
O'SSL-fe51 + Hand-tuned [52]	asm	500
O'SSL-fe51 + OpenSSL-C-OTS [52] + **(ours)**	asm	**500**
O'SSL-fe64 + Hand-tuned [52]	asm	489
Everest-HACL-64 [58]	asm	439
secp256k1		
libsecp256k1 + Rust-EC [29] + **(ours)**	asm	**569**
libsecp256k1 + Fiat Rust [53] + **(ours)**	asm	**535**
libsecp256k1 + Fiat C [54] + **(ours)**	asm	**528**
libsecp256k1 + Fiat C [54]	C	521
libsecp256k1 + Hand-tuned [48]	asm	520
libsecp256k1 + C-OTS [48]	C	517

Note: `-v` indicates the use of vector instructions.

6 Discussion

In this section, we provide further insights on performance comparisons, analysis of potential causes of a performance slowdown, and limitations of the current work.

6.1 Performance Comparison Methodology

We observe that CryptOpt's approach to evaluating the performance of the optimized x86-64 assembly code it generates may introduce some subtle issues. Recall that CryptOpt relies on MeasureSuite [26] for performance comparison between the optimized code generated by CryptOpt and code produced by off-the-shelf compilers. Our further analysis indicates that MeasureSuite uses different types of executable code when comparing execution time. In particular, MeasureSuite compares the execution time of CryptOpt-generated assembly with that of off-the-shelf compiler-generated shared object (as opposed to comparing assembly with assembly).

We note that this comparison method could result in a performance bias. To see this, the shared object files include not only the target function but also other functions to be executed in the machine (such as `_init`, `_fini`, and various linker-generated routines), which could skew performance measurements. While MeasureSuite does attempt to isolate the target function by specifying its address via a function pointer, the overhead from such embedded functions—along with potential alignment, memory layout, and initialization side effects—still influences the measurement accuracy. Furthermore, the CryptOpt-generated Intel-syntax assembly NASM is assembled at runtime by AssemblyLine [27] without invoking external assemblers or linkers, whereas the baseline shared library is produced ahead of time by *GCC* or *Clang* and loaded by the dynamic linker. In other words, the two candidates are created and executed quite differently, which can introduce format-dependent timing effects.

We believed that, to accurately compare the performance, the CryptOpt-generated assembly should be compared against assembly, not shared object (even with the same compiler).

However, as we explained in Chapter 5.1, our re-evaluation indicates that AssemblyLine does not appear to introduce a significant performance gap.

Therefore, we conclude that the performance comparison methodology used in CryptOpt is valid.

6.2 Analysis of the BLS12 Performance

According to the performance comparisons presented in Table 7, our CryptOpt-optimized code via LLVM Bridge outperforms off-the-shelf compiler-generated code with the sole exception of the AUCurves-BLS12. To understand this regression, we conducted a detailed comparative analysis of the generated assembly.

The CryptOpt output contains 1,955 instructions versus 985 for the *llc* `-O3` baseline (i.e., a 2× increase). The stack frame grows from 240 bytes to 2,896 bytes (12×), and spill/reload operations increase from 220 to 750 (3.4×). Critically, 67% of all CryptOpt instructions belong to the MOV family, compared to 51% in the baseline, indicating a higher overhead from data movement relative to arithmetic operations. We identified three primary causes.

First, *implicit operand unawareness*: the MULX instruction requires its multiplicand in `rdx`. CryptOpt handles this reactively—when `rdx` is needed for a new value, the current contents are evacuated

to another register, triggering a cascade of subsequent spills. We traced 28 explicit `rdx` evacuations (versus zero in the baseline), each causing an average of 4.5 downstream spills. This single architectural blind spot accounts for approximately 33% of all spills. The baseline compiler avoids this entirely by scheduling multiplications so that `rdx` values are consumed immediately.

Second, *absent global liveness analysis*: values are spilled without regard for when they will next be needed. We identified instances where values were evicted and then reloaded within 6–20 instructions—lifetimes short enough to remain in registers. We also observed stack-to-stack copies, where a value was loaded from one stack slot, stored to another, and the intermediary register immediately overwritten.

Third, *flag spilling*: approximately 80 carry flags captured via SETC are spilled to single-byte stack slots rather than being consumed immediately through instruction scheduling.

These findings confirm that the JSON conversion discards live-range information, which is critical for register allocation. For primitives with moderate register pressure, e.g., Curve25519, P-448, and secp256k1, CryptOpt's local scheduling heuristics suffice. However, BLS12-381's 384-bit field arithmetic creates sustained pressure across all 16 general-purpose registers, exposing the allocator's limitations. Extending the intermediate representation with explicit last-use annotations and `rdx`-aware scheduling hints would address the identified causes; we estimate such enhancements could eliminate 60–70% of observed spills.

6.3 Limitations with LLVM IR

Our LLVM Bridge targets implementations using unsaturated limb representations, which compile to straightforward LLVM IR patterns that map directly to CryptOpt's instruction set. Extending support to saturated limb representations, such as NIST curves (P-224, P-256, P-384, P-521) and saturated Solinas implementations (e.g., curve25519_solinas), remains an open challenge for future work.

The key distinction lies in how field elements are represented: *Unsaturated representations* use reduced-radix limbs, e.g., 51-bit limbs in 64-bit words. Carries from addition never overflow the word size and are extracted via simple shifts. These implementations compile to straightforward LLVM IR (MUL, LSHR, AND, ADD) that our bridge handles correctly. *Saturated representations*, on the other hand, use full 64-bit limbs. Adding two limbs can overflow, requiring explicit carry-chain operations. At the source code level, these have clear semantics (ADDCARRYX(CARRY_IN, A, B) returning result and carry). However, when compiled to LLVM IR, they decompose into 128-bit arithmetic: operands are zero-extended to `i128`, added, and carries extracted via 64-bit shifts.

Once decomposed, recovering carry-chain semantics through pattern matching is unreliable. LLVM's optimizer reorders operations, interleaves independent carry chains, and produces multiple valid representations for equivalent semantics. This is a fundamental limitation of operating at the LLVM IR level, where domain-specific semantics are lost during compilation.

Implementations using unsaturated representations—including Curve25519, P-448, secp256k1, and Poly1305—are supported. Future work could address saturated representations by modifying source-level carry operations to use overflow-detecting intrinsics, e.g., Rust's `overflowing_add`, which preserve carry semantics through the LLVM compilation pipeline.

7 Conclusion

This thesis presented a framework that extends automatic optimization to a broader range of cryptographic code. We integrated the CryptOpt engine and enhanced it through three distinct contributions: *expansion*, *analysis*, and *methodological validation*.

First, we deconstructed the internal dynamics of the CryptOpt optimizer, which had previously operated as a "black box". By isolating the effects of instruction scheduling and selection, we provided the detailed analysis of its evolutionary mechanism. Our findings reveal that while Decision mutations (M_D) exhibit a higher acceptance rate due to their conservative nature, Permutation mutations (M_P) are the primary driver of performance gains. We further identified that short-distance instruction swaps are more productive than long-distance ones, offering detailed guidance for the design of future search heuristics.

Second, we expanded the applicability of CryptOpt by developing an LLVM Bridge. This translation layer decouples CryptOpt from its dependency on Fiat Cryptography, allowing it to optimize generic LLVM IR derived from widely used languages for cryptographic primitives like Rust and C. This bridge enables developers to produce high-performance, constant-time x86-64 assembly from existing codebases without requiring manual rewrites. Our evaluation demonstrates that this approach is viable, achieving a median speedup of 1.08 times (up to 1.39 times) using our assembly-to-assembly comparison and a median of 1.04 times (up to 1.60 times) with the original measurement framework over off-the-shelf compiler-generated code. However, we identified that while our LLVM Bridge yields competitive results for primitives with moderate register pressure such as Curve25519, P-448, and secp256k1, it faces challenges with complex routines such as BLS12. Our detailed analysis identified three primary causes: implicit operand unawareness causing cascading register spills from `rdx` evacuations, absent global liveness analysis leading to premature value eviction, and inefficient flag spilling. These findings confirm that the JSON conversion discards live-range information critical for register allocation under high pressure.

Third, we investigated the methodology for performance verification. We re-evaluated the original CryptOpt measurements and our results indicate that the use of AssemblyLine does not appear to introduce artificial speedups, validating previous claims. Besides this, we introduced a new assembly-to-assembly comparison framework that ensures a fair evaluation environment without relying on AssemblyLine.

For future work, we plan to address the observed performance bottlenecks by extending the intermediate representation with explicit last-use annotations and `rdx`-aware scheduling hints, which we estimate could eliminate 60–70% of observed spills. Additionally, our current LLVM Bridge supports implementations using unsaturated limb representations; extending support to saturated representations for NIST curves (P-224, P-256, P-384, P-521) remains an open challenge. By addressing these aspects, we aim to further expand the applicability of CryptOpt.

Acknowledgments

I would like to express my deepest gratitude to my supervisors, Chitchanok-sensei and Sako-sensei, for their invaluable guidance and continuous support throughout my master's program. Two years ago, when I visited the University of Melbourne as an exchange student, Sako-sensei introduced me to Chitchanok-sensei, who gave me the opportunity to conduct research on this topic.

Although I initially had little experience in low-level cryptographic implementation and felt overwhelmed by the complexity of CryptOpt, Chitchanok-sensei's consistent support enabled me to make steady progress. Even after I returned to Japan, Chitchanok-sensei continued to follow my work closely and provided valuable feedback. Even though this topic is not Sako-sensei's primary research field, her insightful perspectives and guidance were invaluable to me. I am also deeply grateful to both supervisors for their patience in helping me improve my academic writing and presentation skills. Their careful guidance strengthened my research and its communication, including my submission to CSS 2025. I cannot thank them enough for their unwavering support.

I extend my appreciation to Prof. Yuval for granting me access to his server during this research, and to Dr. Joel for patiently answering my questions regarding the implementation of CryptOpt.

I am also grateful to the researchers in Melbourne Connect, where I spent most of my time while I was in Melbourne. I made many friends and connected with the research community not only in cybersecurity but also in other fields. Their enthusiasm, kindness, and friendship made my time in Melbourne unforgettable.

I also appreciate the members of Sako Lab for their enthusiasm, discussions, and constructive feedback on my research and slides. In particular, my sincere thanks go to Haruki and Takumi for helping refine the English for my publication. I also thank Haruki for joining the CryptOpt project and contributing valuable new insights, and Takumi for our discussions on research topics and coursework assignments. I would also like to thank Watanabe, who sits next to me in the Sako laboratory, for his great motivation toward research and also for sharing insightful perspectives not only on research but also on emerging technologies and our random private chats.

I am grateful to my friends in both Tokyo and Melbourne for their encouragement and for the time we spent together, which helped me stay refreshed. I would also like to thank Himawari, which provides the best meals around our campus and gave me the energy to continue my research with its delicious food.

Finally, I also wish to thank my family for their consistent financial and emotional support throughout my master's program.

Overall, my master's program has been a wonderful experience, and I am sincerely grateful to everyone who supported me along this journey.

References

[1] D. J. Bernstein, T. Chou, and P. Schwabe, "McBits: fast constant-time code-based cryptography," in *CHES*, vol. 8086, 2013, pp. 250–272.

[2] D. J. Bernstein, C. Chuengsatiansup, and T. Lange, "Curve41417: Karatsuba revisited," in *CHES*, 2014, pp. 316–334.

[3] D. J. Bernstein, C. Chuengsatiansup, T. Lange, and P. Schwabe, "Kummer strikes back: New DH speed records," in *ASIACRYPT*, 2014, pp. 317–337.

[4] D. J. Bernstein, C. Chuengsatiansup, T. Lange, and C. van Vredendaal, "NTRU Prime: reducing attack surface at low cost," in *SAC*, 2017, pp. 235–260.

[5] T. Chou, "Sandy2x: new Curve25519 speed records," in *SAC*, 2015, pp. 145–160.

[6] ——, "QcBits: constant-time small-key code-based cryptography," in *CHES*, vol. 9813, 2016, pp. 280–300.

[7] C. Chuengsatiansup, M. Naehrig, P. Ribarski, and P. Schwabe, "PandA: pairings and arithmetic," in *Pairing*, vol. 8365, 2013, pp. 229–250.

[8] C. Chuengsatiansup, and D. Stehlé, "Towards practical GGM-Based PRF from (module-) learning-with-rounding," in *SAC*, 2019, pp. 693–713.

[9] M. J. Kannwischer, J. Rijneveld, and P. Schwabe, "Faster multiplication in $Z_{2^m}[x]$ on Cortex-M4 to speed up NIST PQC candidates," in *ACNS*, 2019, pp. 281–301.

[10] A. Pnueli, M. Siegel, and E. Singerman, "Translation validation," in *TACAS*, 1998, pp. 151–166.

[11] J. Bosamiya, S. Gibson, Y. Li, B. Parno, and C. Hawblitzel, "Verified transformations and Hoare logic: Beautiful proofs for ugly assembly language," in *VSTTE*, 2020, pp. 106–123.

[12] F. Recoules, S. Bardin, R. Bonichon, L. Mounier, and M.-L. Potet, "Get rid of inline assembly through verification-oriented lifting," in *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019, pp. 577–589.

[13] M. Schoolderman, J. Moerman, S. Smetsers, and M. C. J. D. van Eekelen, "Efficient verification of optimized code - correct high-speed X25519," in *NFM*, 2021, pp. 304–321.

[14] J. Filliâtre, and A. Paskevich, "Why3 - where programs meet provers," in *ESOP*, 2013, pp. 125–128.

[15] Y. Chen, C. Hsu, H. Lin, P. Schwabe, M. Tsai, B. Wang, B. Yang, and S. Yang, "Verifying Curve25519 software," in *CCS*, 2014, pp. 299–309.

[16] M. Tsai, B. Wang, and B. Yang, "Certified verification of algebraic properties on low-level mathematical constructs in cryptographic programs," in *CCS*. ACM, 2017, pp. 1973–1987.

[17] A. Polyakov, M. Tsai, B. Wang, and B. Yang, "Verifying arithmetic assembly programs in cryptographic primitives (invited talk)," in *CONCUR*, ser. LIPIcs, vol. 118. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018, pp. 4:1–4:16.

[18] Y. Fu, J. Liu, X. Shi, M. Tsai, B. Wang, and B. Yang, "Signed cryptographic program verification with typed cryptoline," in *CCS*. ACM, 2019, pp. 1591–1606.

[19] A. Abdulrahman, H. Becker, M. J. Kannwischer, and F. Klein, "Fast and clean: Auditable high-performance assembly via constraint solving," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2024, no. 1, pp. 87–132, 2024.

[20] J. Kuepper, A. Erbsen, J. Gross, O. Conoly, C. Sun, S. Tian, D. Wu, A. Chlipala, C. Chuengsatiansup, D. Genkin, M. Wagner, and Y. Yarom, "CryptOpt: verified compilation with randomized program search for cryptographic primitives," *Proc. ACM Program. Lang.*, vol. 7, no. PLDI, pp. 1268–1292, 2023.

[21] A. Erbsen, J. Philipoom, J. Gross, R. Sloan, and A. Chlipala, "Simple high-level code for cryptographic arithmetic - with proofs, without compromises," in *IEEE Symposium on Security and Privacy*. IEEE, 2019, pp. 1202–1219.

[22] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. CRC Press, 1996.

[23] ,*Intel® 64 and IA-32 Architectures Optimization Reference Manual Volume 1*, Intel Corporation, Sep. 2023. [Online]. Available: https://www.intel.com/content/www/us/en/content-details/671488/intel-64-and-ia-32-architectures-optimization-reference-manual-volume-1.html

[24] ,*Intel® 64 and IA-32 Architectures Software Developer's Manual Volume 3 (3A, 3B, 3C, & 3D): System Programming Guide*, Intel Corporation, Oct. 2025. [Online]. Available: https://cdrdv2.intel.com/v1/dl/getContent/671447

[25] A. Auger, and B. Doerr, Eds., *Theory of Randomized Search Heuristics: Foundations and Recent Developments*, ser. Series on Theoretical Computer Science. World Scientific, 2011, vol. 1.

[26] 0xADE1A1DE, "Measuresuite v2.2.2." https://github.com/0xADE1A1DE/MeasureSuite, 2023, accessed: 2025-02-01.

[27] ——, "Assemblyline v1.3.2." https://github.com/0xADE1A1DE/AssemblyLine, 2023, accessed: 2025-02-01.

[28] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, "RustBelt: securing the foundations of the Rust programming language," in *Proceedings of the ACM on Programming Languages*, vol. 2. ACM journals, 2018, pp. 1–34.

[29] R. Developers, "Rustcrypto," https://github.com/RustCrypto, 2020, accessed: 2025-04-05.

[30] Rustls Developers, "rustls: A modern TLS library in Rust," https://github.com/rustls/rustls, 2025, accessed: 2025-05-07.

[31] Rust Project Developers, "Rust Compiler Development Guide," https://rustc-dev-guide.rust-lang.org/overview.html, 2025, accessed: 2025-05-07.

[32] llvm-admin team, "The LLVM Compiler Infrastructure Project," https://llvm.org/, 2025, accessed: 2025-05-07.

[33] P. C. Kocher, "Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems," in *CRYPTO*, ser. Lecture Notes in Computer Science, vol. 1109. Springer, 1996, pp. 104–113.

[34] S. Huang, R. Q. Sim, C. Chuengsatiansup, Q. Guo, and T. Johansson, "Cache-timing attack against HQC," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2023, no. 3, pp. 136–163, 2023.

[35] S. H. W. Cheng, C. Chuengsatiansup, D. Genkin, D. McNeil, T. Murray, Y. Yarom, and Z. Zhang, "Evict+spec+time: Exploiting out-of-order execution to improve cache-timing attacks," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2024, no. 3, pp. 224–248, 2024.

[36] P. C. Kocher, J. Jaffe, and B. Jun, "Differential power analysis," in *CRYPTO*, ser. Lecture Notes in Computer Science, vol. 1666. Springer, 1999, pp. 388–397.

[37] S. Mangard, E. Oswald, and T. Popp, *Power analysis attacks - revealing the secrets of smart cards*. Springer, 2007.

[38] M. Brinkmann, C. Chuengsatiansup, A. May, J. Nowakowski, and Y. Yarom, "Leaky mceliece: Secret key recovery from highly erroneous side-channel information," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2025, no. 2, pp. 94–125, 2025.

[39] K. Gandolfi, C. Mourtel, and F. Olivier, "Electromagnetic analysis: Concrete results," in *CHES*, ser. Lecture Notes in Computer Science, vol. 2162. Springer, 2001, pp. 251–261.

[40] J. Heyszl, S. Mangard, B. Heinz, F. Stumpf, and G. Sigl, "Localized electromagnetic analysis of cryptographic implementations," in *CT-RSA*, ser. Lecture Notes in Computer Science, vol. 7178. Springer, 2012, pp. 231–244.

[41] J. Jancar, M. Fourné, D. D. A. Braga, M. Sabt, P. Schwabe, G. Barthe, P.-A. Fouque, and Y. Acar, ""they're not that hard to mitigate": What cryptographic library developers think about timing attacks," in *IEEE Symposium on Security and Privacy*. IEEE, 2022, pp. 632–649.

[42] T. dudect team, "dudect: Detecting constant-time violations in assembly code," https://github.com/oreparaz/dudect.

[43] B. L. Welch, "The generalization of "student's" problem when several different population variances are involved," *Biometrika*, vol. 34, no. 1/2, pp. 28–35, 1947.

[44] "Intel performance hybrid architecture & software optimizations, part one: Introduction to performance hybrid architecture," Intel Corporation, Tech. Rep. 349168, 2021, white Paper.

[45] "Intel performance hybrid architecture & software optimizations, part two: Developing for intel performance hybrid architecture," Intel Corporation, Tech. Rep. 349169, 2021, white Paper.

[46] AU-COBRA, "Aucurves - high assurance cryptography by means of code synthesis." https://github.com/AU-COBRA/AUCurves, 2022, accessed: 2025-02-01.

[47] D. J. Bernstein, and T. Lange, "eBACS: ECRYPT Benchmarking of Cryptographic Systems," http://bench.cr.yp.to/supercop/supercop-20221005.tar.xz, 2022, accessed: 2025-07-10.

[48] B. Core, "libsecp256k1 - optimized c library for ecdsa signatures and secret/public key operations on curve secp256k1," https://github.com/bitcoin-core/secp256k1/blob/9526874d1406a13193743c605ba64358d55a8785/src/field_5x52_int128_impl.h, 2021.

[49] mit plv, "A work-in-progress language and compiler for verified low-level programming," https://github.com/mit-plv/bedrock2, 2018, accessed: 2025-04-05.

[50] dalek-cryptography Developers, "Dalek elliptic curve cryptography," https://github.com/dalek-cryptography/curve25519-dalek, 2023, accessed: 2025-04-05.

[51] H. de Valence, I. Lovecruft, and T. Arcieri, "The ristretto group," https://ristretto.group/ristretto.html, n.d., accessed: 2025-05-03.

[52] "OpenSSL: Cryptography and SSL/TLS Toolkit," https://github.com/openssl/openssl, commit snapshot accessed 2 July 2025. The evaluation uses the files `crypto/ec/curve25519.c` (portable C) and `crypto/ec/asm/x25519_x86_64.pl` (hand-optimized x86 – 64).

[53] T. F. C. Contributors, "Fiat-Crypto rust bindings," https://github.com/mit-plv/fiat-crypto/tree/main/rust, 2023, commit `Commit 4e7dde9` (includes Curve25519, P448, Secp256k1, Poly1305).

[54] ——, "Fiat-Crypto c bindings," https://github.com/mit-plv/fiat-crypto/tree/main/c, 2023, commit `Commit c79cf60` (includes Curve25519, P448, Secp256k1, Poly1305).

[55] M. Dempsky, "Curve25519 reference implementation," SUPERCOP cryptographic benchmarking suite, 2008, derived from public domain code by D. J. Bernstein, version 20081011. [Online]. Available: https://bench.cr.yp.to/supercop.html

[56] D. J. Bernstein, "Curve25519 reference implementation (ref10)," SUPERCOP cryptographic benchmarking suite, 2006, reference implementation by the original designer of Curve25519. [Online]. Available: https://bench.cr.yp.to/supercop.html

[57] A. Langley, "Curve25519-donna," https://github.com/agl/curve25519-donna, 2022.

[58] "HACL," https://github.com/hacl-star/hacl-star, 2022.

[59] M. A. Bokhari, B. Alexander, and M. Wagner, "Towards rigorous validation of energy optimisation experiments," in *GECCO*, 2020, pp. 1232–1240.

A Performance Measurement in CryptOpt

Note that in our measurements, we consider three different assembly code: (1) CryptOpt-optimized assembly, (2) baseline x86-64 assembly in GAS format, and (3) baseline x86-64 assembly in NASM format.

Algorithm 1 is our algorithm for performance measurement and correctness verification. In addition to the three assembly implementations (*gas_func*, *nasm_func*, *cryptopt_func*), the algorithm also requires the following inputs:

- *bound*: the maximum input bound
- *size*: the length of the input and output arrays
- *batch_size*: the number of executions in each batch
- *nob*: the number of batches

We note that *bound* and *size* vary depending on the underlying finite-field arithmetic. However, *batch_size* and *nob* are independent of the specific implementation.

A.1 Batching

The performance measurement in CryptOpt includes three strategies to handle challenges in measuring time.

First of all, recall that cycle counters possess a fixed granularity of at best one cycle. This granularity presents an accuracy problem when measuring functions of vastly different lengths, for example, ranging from under 100 to approximately 2000 instructions. CryptOpt copes with this problem by measuring functions in *batches*, consisting of multiple executions of the same function. The *batch size* is dynamically adjusted for each function: smaller functions use a larger batch size, while longer functions use a smaller batch size. The adjustment mechanism targets a *cycle goal* of around $10,000$ cycles per batch measurement. This target ensures a relatively consistent measurement accuracy (approximately four to five significant digits) across all function sizes. Crucially, this dynamic adjustment of the batch size is performed independently on each hardware platform before every measurement. This approach compensates for platform-specific interpretations of "cycles" and varying CPU boost behaviors, enabling comparable accuracy across different functions and hardware.

Secondly, hardware measurements are susceptible to noise originating from sources such as operating system interrupts, process scheduling, or thermal throttling. While batching helps with granularity, it can potentially amplify the noise captured due to longer measurement intervals. To mitigate this issue, each batch is measured multiple times consecutively, defined by the *number of batches*. Empirically, the number of batches of 31 appears to work well. The median value of the number of batch measurements is used as the final performance figure for the function. Compared to other methods, using the median is found to be not only effective in filtering outliers but also yields results with no differences in statistical significance.

Finally, modern processor features such as speculative execution, branch prediction, and caching, can introduce measurement biases favoring certain code structures. These issues are mitigated by randomizing the sequence in which different function batches are measured, following a technique referred to as R3-validation [59].

Algorithm 1 Measurement of Three Functions

```
function MEASURE_THREE_FUNCTIONS(bound, size, gas_func, nasm_func, cryptopt_func, batch_size, nob)
    cycles_gas ← empty list
    cycles_nasm ← empty list
    cycles_cryptopt ← empty list
    for i ← 1 to nob do
        funcs ← [gas_func, nasm_func, cryptopt_func]
        SHUFFLE(funcs)                                  ▷ Randomly shuffle the execution order
        in0 ← GENERATE_RANDOM_LOOSE_INPUT(bound, size)
        in1 ← GENERATE_RANDOM_LOOSE_INPUT(bound, size)
        for all f ∈ funcs do
            c ← MEASURE_ONE_BATCH(f, out, in0, in1, batch_size)
            if f = gas_func then
                APPEND(cycles_gas, c)
                COPY(out_gas, out)
            else if f = nasm_func then
                APPEND(cycles_nasm, c)
                COPY(out_nasm, out)
            else
                APPEND(cycles_cryptopt, c)
                COPY(out_cryptopt, out)
            end if
        end for
        if out_gas ≠ out_nasm or out_gas ≠ out_cryptopt then
            error: "Output mismatch detected"
        end if                                          ▷ Check the correctness of the outputs
    end for
    gas_median ← MEDIAN((cycles_gas))
    nasm_median ← MEDIAN((cycles_nasm))
    cryptopt_median ← MEDIAN((cycles_cryptopt))
    return (gas_median, nasm_median, cryptopt_median)
end function
```

A.2 Measurement with Dynamic Batch Size

Following the methodology of CryptOpt [20], we extend Algorithm 1 with per-measurement dynamic batch size adjustment. Each function maintains its own batch size, adjusted after every measurement:

1. Start with an initial batch size $bs = 200$ for each function
2. After each measurement, adjust the batch size:

$$bs \leftarrow \left\lceil \frac{cyclegoal}{PC} \times bs \right\rceil$$

   where PC is the measured cycles and $cyclegoal = 10,000$
3. Clip to the valid range: $bs \leftarrow \min(10000, \max(5, bs))$
4. Use the new batch size for the next measurement

This per-measurement adjustment maintains approximately 10,000 cycles per batch throughout the entire measurement process, thus ensuring consistent precision under varying system conditions. For example, if a function measures 4,500 cycles with $bs = 200$, the next measurement uses $bs = \lceil 10000/4500 \times 200 \rceil = 445$.

This ensures that each measurement achieves comparable statistical precision regardless of the performance characteristics of the code being measured.

A.3 Bet-and-Run Parameters

For the Bet-and-Run [20] parameters, we adopt the same setting as CryptOpt [20] as it strikes a balance between exploring diverse starting points and exploiting promising regions of the search space. With a total computational budget of $200{,}000$ mutation evaluations, the optimizer first generates 20 initial candidates (`bets`), allocating $1{,}000$ evaluations to each, thereby spending $20 \times 1{,}000 = 20{,}000$ mutations in the bet phase. This corresponds to a bet-phase ratio of $20{,}000/200{,}000 = 0.1$ (10% of the entire budget). The remaining $200{,}000 - 20{,}000 = 180{,}000$ mutation evaluations constitute the run budget reserved for the final optimization phase.

B CSS 2025 Paper

This appendix includes the author's CSS 2025 conference paper. The original version contained two issues: (1) the performance measurements were conducted on E-Cores instead of P-Cores, and (2) the measurement algorithms were incorrectly implemented. The following is the corrected version of the paper, with measurements re-run on P-Cores and the implementation algorithms fixed.

暗号コード自動最適化フレームワーク CryptOpt の拡張

木村 悠人[1,a)]　Chitchanok Chuengsatiansup[2]　佐古 和恵[1]

概要：暗号ソフトウェアを高速化するために最適化は必要である．汎用コンパイラは一定の最適化を行えるものの，ドメイン固有の最適化を十分に活用できず，専門家が手作業で書いたアセンブリほど高性能なコードを生成できない場合がある．これに対し CryptOpt(Kuepper ほか, PLDI 2023) は，セキュリティ要件に起因し，暗号コードに共通して現れる実装パターンを最適化過程に取り込み，専門家が手書きで生成するアセンブリと同程度の性能を達成した．しかし，CryptOpt には入力フォーマットが限定されているという制約がある．本稿では，CryptOpt を拡張し，二つの主要な側面で強化した．第一に，LLVM 中間表現（LLVM IR）を入力として受け取り，CryptOpt の最適化エンジンが扱える形式へ変換するコード変換手法を設計・実装した．LLVM IR は多数のプログラミング言語で広く使用されているため，本拡張により CryptOpt は多様な言語のコードを最適化できるようになる．第二に，既存の CryptOpt の性能評価手法を改良し，測定バイアスを低減した．ベンチマークの結果，LLVM IR を経由して CryptOpt が最適化したコードは，汎用コンパイラが生成したコードに対し最大 1.39 倍の性能向上を示した．

キーワード：暗号，最適化，コード生成

On Enhancing the Automatic Cryptographic Code Optimization Framework CryptOpt

Haruto Kimura[1,a)]　Chitchanok Chuengsatiansup[2]　Kazue Sako[1]

Abstract: High-speed cryptographic software is greatly desirable. While off-the-shelf compilers can perform decent optimization, they may fail to benefit from domain-specific optimization, thus not producing as performant code as hand-optimized assembly written by experts in the field. To this end, CryptOpt (Kuepper et al., PLDI 2023) tackles this challenge by integrating coding styles, which often appear in cryptographic code due to security requirements, as part of the optimization. As a result, code produced by CryptOpt achieves competitive performance as hand-optimized code. Despite an impressive outcome, CryptOpt has a limitation on its restrictive input format. This paper enhances the automatic cryptographic code optimization CryptOpt by presenting two contributions. First of all, we design and implement code transformations, which, upon receiving LLVM intermediate representation (LLVM-IR) input, convert it into a format that is applicable to CryptOpt's optimizer engine. Since LLVM IR is widely and commonly used by a large number of programming languages, this means that this enhancement enables CryptOpt to optimize code from a broad range. Secondly, we improve the performance evaluation method to reduce potential bias in the measurement. According to our benchmarking results, our LLVM IR CryptOpt-optimized code achieves speedups of up to 1.39 times over off-the-shelf compiler-generated code.

Keywords: Cryptography, Optimization, Code Generation

[1] 早稲田大学
Waseda University
[2] The University of Klagenfurt

[a)] haruto1999@akane.waseda.jp

1. はじめに

暗号は機微なデータを保護するための仕組みである．暗号ソフトウェアが広範かつ集中的に利用される現代のデジタル環境において，高効率な実装への需要は尽きることがない．暗号プリミティブの多様性，複数の安全性水準の選択肢は，それだけで実装者に困難を課す．加えて，これらの実装は最終的に実機上で動作するため，異なるハードウェアアーキテクチャやデバイス側の制約が，安全かつ高効率な暗号ソフトウェアの実装をさらに難しくする．

可搬性に対処する一つの方法は，C や Rust のような高水準言語で暗号ソフトウェアを実装し，ターゲットプラットフォーム上で動作可能な機械語へコンパイルすることである．この方法は，対象プラットフォームの変化（例えば新世代機の導入）に伴って再実装する必要が生じる問題を回避できる. しかし，そのようなコードの性能は，専門家が直接アセンブリで実装したコードに比べて遅いことが多い [1], [2], [3], [4]．しかも，少数の専門家の手作業による最適化に依拠する開発体制は，暗号ソフトウェア需要の増大に伴ってスケールしない．加えて，手作業の実装はバグを導入する可能性がある．

実装が意図どおりに動作し期待どおりの結果を生成しているかを検証する観点では，さまざまなツールと技法が利用可能である [5], [6], [7]．同様に，暗号用コードの実装および最適化のスケーラビリティに対処する提案も多数存在し，形式的検証と統合されたものもある．

その一つとして CryptOpt [8] がある．これは最適化過程において革新的なアルゴリズムを用いる自動暗号化最適化ツールである．CryptOpt [8] は効率的実装，タイミングサイドチャネル耐性，および実装検証という三つの問題を同時に解決することに成功している．もっとも，CryptOpt の利用可能性には制約があり，比較的限定的な入力空間を要求する．具体的には CryptOpt の入力形式は Fiat Cryptography [9]（関数型プログラムをコードへ翻訳し両者の同値性を証明する枠組み）を前提として設計されており，主に Fiat 由来の入力を受け付ける．したがって，Fiat 形式ではないコードを CryptOpt で最適化することはできない．Fiat Cryptography 以外を最適化できることを示す試みはあるものの，非 Fiat 実装の最適化は，筆者の知る限り，Bitcoin Core の secp256k1 に対する C 実装を対象にしたもののみである [8].

この制約を克服するため，本研究は CryptOpt をより広範な実装に適用可能にする枠組みを導入する．具体的には LLVM ブリッジと名付けた経路を設計・実装し，Rust や C の実装を LLVM 中間表現（LLVM IR）へコンパイルしたのち，それを CryptOpt に適した入力形式である JSON 形式へ変換する．さらに，提案方式が出力したコードと汎用コンパイラが生成したコードの性能を P-Core 上で比較した．13 実装・計 25 関数を本 LLVM ブリッジ経由で CryptOpt に入力して評価した結果，CryptOpt 生成コードは汎用コンパイラ（*-O3*）を多くのケースで上回り，最大 1.39 倍の性能向上を達成した．

本稿の貢献

本稿の貢献は以下のとおりである．

- **汎用 LLVM ブリッジの実装**．*clang* 及び *rustc* が出力する LLVM 中間表現（LLVM IR）を，CryptOpt が期待する JSON 形式へ変換するフロントエンドを実装した．

- **公平な性能評価と改善**．CryptOpt の共有オブジェクト（.so）に対するベンチマークをやめ，アセンブリ対アセンブリの直接比較に置き換え，フォーマットの差異による測定ノイズを低減した．この公平な評価の下，P-Core 上で C，Rust 双方における 7 つの実用プリミティブ（Curve25519，P448，secp256k1，Poly1305，BLS12 に加えライブラリ実装 2 種）ならびに OpenSSL 実装に対して，汎用コンパイラが最高レベル最適化（*-O3*） を適用して生成したコードに対し，最大 1.39 倍の高速化を達成した．なお，この数値は評価法の変更による見かけの改善ではなく，CryptOpt が生成した命令列（instruction sequence；本稿では LLVM IR/アセンブリの *instruction* を以下 “命令” と呼ぶ）そのものの優位に起因する．

2. 背景

本節では，本研究に関連する先行研究と概念として Fiat Cryptography と CryptOpt そして LLVM について概説する．

2.1 Fiat Cryptography

Fiat Cryptography [9] は，暗号アルゴリズムの数学的仕様から暗号コードを自動生成するためのフレームワークである．具体的に，入力として Coq 定理証明支援系内の関数型プログラムとして記述された有限体演算の高水準形式仕様が定められている．Fiat Cryptography は，これらの仕様から低水準コードを合成すると同時に，生成コードが元の数学的定義を正しく実装していることを証明する．この過程では，演算列を記述する中間表現である Fiat IR を生成するのが通例である．Fiat IR からは，C，Rust，Zig などのコードを生成することができる．

2.2 CryptOpt

CryptOpt [8] は, 暗号プリミティブ向けに高性能な x86-64 アセンブリコードを生成するコード生成ライブラリであり，想定されるプロセッサアーキテクチャに合わせて最適化さ

れる．また，Fiat Cryptography フレームワークを活用しているため，生成されたコードの正当性保証も与えられる．

図 1 は Fiat Cryptography と CryptOpt の連携を概観する．CryptOpt は Fiat Cryptography が生成する低水準中間表現 Fiat IR を受け取る．そこから CryptOpt は二つの中間表現を生成する．(1) 性能評価に用いる Fiat C （Fiat Cryptography の C 実装）と，(2) CryptOpt が最適化の過程で用いる Fiat JSON である．Fiat JSON は Fiat IR の全情報—演算，入出力のビット幅，境界（bounds）—を保持する JSON 構造である．この JSON 入力は前処理を経て CryptOpt の内部モデル（CryptOpt IR ）に変換され，演算列，順序，変異の決定が保持される．

また，CryptOpt には，C で実装された Bitcoin Core の secp256k1 を CryptOpt の入力形式に変換するための“Bitcoin Core ブリッジ”を実装している．図 1 の青矢印にあるように，まず Bitcoin Core は *Clang*（LLVM プロジェクトの C/C++ 用コンパイラフロントエンド）を用いて LLVM IR にコンパイルされる．その後，LLVM IR の各種演算などの関連情報を抽出し，CryptOpt に適した中間 JSON 形式へ変換する．この JSON は Fiat JSON と同趣旨で本質的な演算を記述するが，Coq による形式証明を経ずに LLVM IR から直接得られる点が Fiat Cryptography を入力とした場合と異なる．この中間 JSON はさらに CryptOpt の内部表現へと翻訳され，最適化の入力となる．

最適化戦略として，CryptOpt はランダム探索を用いる．あらかじめ定義されたヒューリスティックや抽象的性能モデルに依存するのではなく，正当な初期実装に対して命令差し替えや順序入れ替えといった変異を反復的に適用する．同時に，元の実装と変異後の実装の実行時間を対象ハードウェア上で直接計測し，速い方を残す．これにより，有限の計算資源内で機械語実装の探索空間を効率的に探索でき，ハードウェア世代やマイクロアーキテクチャの違いにも自動的に対応する．

正当性の保証については，CryptOpt が利用する性能計測ツール MeasureSuite [10] に含まれるプログラム同値性チェッカーを用いる．MeasureSuite は複数の入力プログラムを受け取り，それらの性能を比較するとともに，プログラムの出力がランダムな入力に対し一致するかを確認する．CryptOpt の場合には，性能比較のためのベースライン実装（例．Fiat C ）からコンパイルした共有オブジェクト（`.so`）と，CryptOpt が生成した最適化済みアセンブリ（`.asm`）を MeasureSuite の入力とする．

性能ベンチマーク時には，CryptOpt はベースラインとの比較用に，参照実装である Fiat C や Bitcoin Core からコンパイルした共有オブジェクト（`.so`）を用いる．一方で CryptOpt の出力はアセンブリ（`.asm`）であるため，共有オブジェクト（`.so`）のベースラインと比較するために，MeasureSuite に統合された AssemblyLine [11] を用いて CryptOpt 生成の アセンブリ（`.asm`）をオブジェクト（`.o`）へ変換する．計測対象の関数（乗算や二乗算）について，AssemblyLine が生成したオブジェクト（`.o`）内の該当関数と，ベースラインの共有オブジェクト（`.so`）内の対応関数の実行時間を比較し，性能を評価する．

2.3 LLVM

LLVM プロジェクトは，モジュール式で再利用可能なコンパイラおよびツールチェーン技術の集合体である．本研究に関連する中核要素は LLVM 中間表現（LLVM IR）である．LLVM IR は言語非依存の低水準中間表現であり，コンパイラのフロントエンドが出力する共通対象であると同時に，最適化パスやコード生成バックエンドへの入力として設計されている [12].

LLVM IR は静的単一代入（Static Single Assignment; SSA）形式を用いる．すなわち各レジスタ（仮想レジスタ）は定義されてから使用されるまで，値の代入は必ず一度だけされるため，多様な解析や変換が簡素化される．C/C++ に対する *Clang* や Rust に対する *rustc* を含む各種高水準言語のコンパイラは，ソースコードから LLVM IR を生成できる．その後，LLVM はこの共通 IR 上で動作する最適化パス群を提供し，最終的にバックエンドが最適化済み IR を x86-64 など多様な対象アーキテクチャ向けの機械語へと翻訳する．

3. 本提案のアプローチ

本研究の第一の目的は，Fiat Cryptography [9] 由来の入力仕様に限らず，より広範な暗号コードを最適化できるように CryptOpt [8] の能力を拡張することにある．CryptOpt は主に Fiat の中間表現（Fiat IR ）を入力として用いるため，我々の主要課題は Fiat IR 以外の入力を CryptOpt に受理させることである．この目的のために，我々は変換系列を設計・実装し，最終的に CryptOpt と互換な形式へ到達させる．

我々は，とりわけ各種高水準言語で広く用いられている LLVM 中間表現（LLVM IR）を CryptOpt の入力として受け付けられるようにすることに焦点を当てる．フレームワークの概略は図 2 に示す．以降，図 2 に示した一連の変換手順を “パイプライン” と呼ぶ．本研究の中核的な困難は，汎用で低水準な LLVM IR と，有限体演算に特化した CryptOpt の内部モデルとの間にある意味的ギャップを橋渡しする点にある．

なお，原著の CryptOpt 論文 [8] では，Fiat Cryptography に由来しない入力として Bitcoin Core の secp256k1 曲線のベンチマークも含まれている．しかし，CryptOpt が扱えた非 Fiat 入力はこの一例にとどまる．我々の調査によれば，LLVM IR のようなより汎用的な入力を直接扱う能力には顕著な制約が存在する．

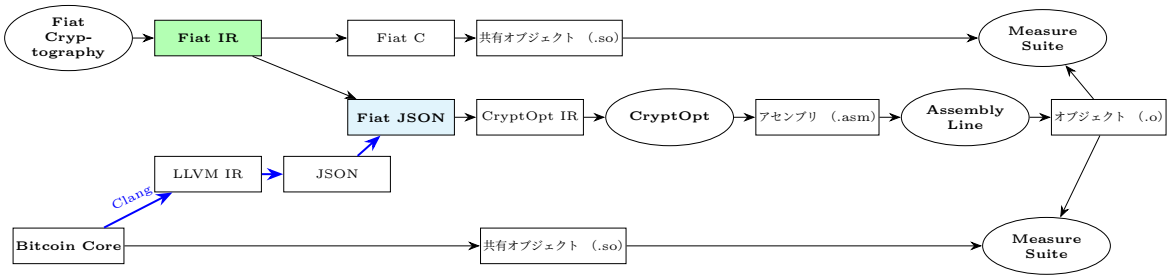

図 1　オリジナルの CryptOpt ワークフローの概観

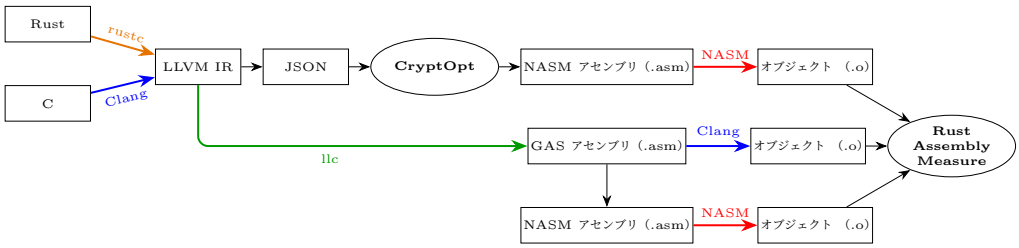

図 2　本研究の概観

3.1　フォーマット変換

入力として LLVM IR を受け付けることを目標とするため，Rust や C といった高水準言語で実装された暗号コードは，まず LLVM IR へ変換する必要がある．この段階は汎用コンパイラを用いればよい．具体的には，Rust 実装には *rustc*，C 実装には *Clang* を用いる．

LLVM IR を取得した後の工程で，しかも最大の難所は，この低水準の LLVM IR を CryptOpt が扱える JSON 形式へ変換することだ．そのために，LLVM IR を解析し，正規表現を用いて各命令の構成要素を抽出する．抽出した要素は `name`，`operation`，`modifiers`，`datatype`，`arguments` の五つに分類する．`name` は変数名，`operation` は演算種別，`modifiers` は最適化や意味上のヒント，`datatype` は符号付き整数の型，`arguments` は被演算子を格納する．

CryptOpt が受理可能な JSON を構成するには，JSON に含まれる各操作が CryptOpt が理解・生成できる命令の集合でなければならない．この要件を満たすため，既存の CryptOpt 命令だけでは不十分となる場合があることを踏まえ（たとえば AUCurves (BLS12) [13] を入力とする場合），CryptOpt の対応命令を拡張した．

この JSON は共通の中間表現として機能し，CryptOpt の Bitcoin Core ブリッジ に接続して，最終的に CryptOpt が受理する中間表現へと変換できる（詳細は appendix A.1）．

3.2　境界チェックと panic ブロック

Rust のような安全志向言語から得られる LLVM IR には，境界チェックや panic ブロック（境界違反などのエラー時に例外処理へ分岐して異常終了するための基本ブロック）が含まれる場合がある．これらは実装中の潜在的なエラーに対処するために重要だが，入力が妥当でエラーが発生しないことを前提とする限り，コードの機能自体には直接影響しない．したがって，本研究のフォーマット変換では，LLVM IR 中の境界チェック分岐と panic ブロックを抽出対象から除外し，後続の変換・最適化でも扱わない．

なお，Fiat を入力源とする元の CryptOpt [8] においても，ソースレベルでは境界チェックを含んでいても，コンパイラ最適化が不要とみなした定義を除去する傾向にあるため，LLVM IR 段階では当該チェックが失われることがある．要するに，LLVM IR に見られる panic ブロックの存在は，ソース言語とコンパイラ設定に起因するアーティファクトである．

3.3　変数名の再命名とアドレス規約の整合

LLVM IR は上述の通り SSA 形式であるが，変数名は必ずしも連番形式でない．Bitcoin Core ブリッジは，`x0`，`x1`，`x2` のような連番の変数名から恩恵を受ける．一方で，Rust 実装から生成される LLVM IR はこの連番規約に必ずしも従わず，`%0.i`，`%0.i14`，$\%_{\mathtt{x}}\mathtt{1}$，$\%_{\mathtt{0}}\mathtt{.i2}$，`%11` のように直感的でない不規則な名前が現れることがある．そこで本パイプラインでは，後段の処理を簡潔にするため，変数名を整理して連番形式へと再命名する．

加えて，LLVM IR と CryptOpt のあいだでアドレス表現の規約不一致も解消する．LLVM IR はオフセットをバイト単位で数えるのに対し，CryptOpt はワード（64 ビット）単位で扱う．この差異を吸収するため，LLVM IR 側のバイトオフセットをワードインデックスへ変換し，バイト基準のメモリ意味論とワード基準のアドレッシングの齟齬を整合させる．

3.4 対応命令集合の拡張

CryptOpt は Fiat IR を主な入力としている．その結果，素の LLVM IR の各種命令は原則としてサポート外である．

本研究のパイプラインでは，LLVM IR を CryptOpt 用の JSON 形式へ変換する過程（section 3.1）で，CryptOpt が未対応の命令である `sext`（符号拡張）と `select` が現れる．この問題に対処するため，これら未対応命令を CryptOpt 互換のプリミティブへ写像する．具体的には，それぞれを CryptOpt がサポートする `cmovznz`（ゼロ/非ゼロ条件付き移動）へ写像する設計とした．以下に写像手順の詳細を述べる．

3.4.1 `select` から `cmovznz` への写像

LLVM IR の `select` は，分岐を用いずに真偽条件に基づいて二つの値のいずれかを選ぶ命令である．CryptOpt の `cmovznz`（ゼロ/非ゼロ条件付き移動）は同等の機能を提供するが，明示的なブールではなく整数フラグに基づいて動作する．`select` の例は次のとおりである．

```
<result> = select i1 <cond>, i64 <val1>, i64
                    <val2>
```

すなわち，条件 `<cond>` が真（値 1）なら `<result>` に `<val1>` を代入し，そうでなければ `<val2>` を代入する．CryptOpt 側への写像では，`<cond>` を 0 または 1 の整数フラグとして解釈する．`cmovznz` はフラグがゼロか非ゼロかに応じて二つの値から選択する．等価な変換は次のとおりである．

```
<result> = cmovznz i1 <cond>, i64 <val2>, i64
                    <val1>
```

ここで `<cond>` が真（非ゼロ）なら `<val1>` が選ばれ，偽（ゼロ）なら `<val2>` が選ばれる．`cmovznz` は「フラグが非ゼロなら第 2 オペランド」を選ぶ命令であり，`<val1>` と `<val2>` の位置を入れ替えるため，`select` と同値の処理となる．LLVM の `i1` ブールは 1 ビット整数（0/1）として安全に扱えること，および `cmovznz` が分岐なしに同じ二者択一を行うことから，本変換はプログラムの論理を保ったまま安全に適用できる．

3.4.2 `sext` から `cmovznz` への写像

LLVM IR の `sext` は値のビット幅を増やしつつ符号を保存する命令である．たとえば 1 ビットのブール（`i1`）を 64 ビット整数（`i64`）へ拡張する場合，入力が真（1）なら出力は `-1`（`0xFFFFFFFFFFFFFFFF`）になり，偽（0）なら出力は `0`（`0x0000000000000000`）になる．`sext` の例は次のとおりである．

```
<result> = sext i1 <val3> to i64
```

これは「`<val3>` が真なら上位ビットを 1 で埋めた結果（すなわち `-1`），偽なら `0`」という意味である．この機能は CryptOpt の `cmovznz` でも実現できる．`<val3>` を条件フラグとして用い，ゼロか非ゼロかで二つの定数を選べばよい．すなわち次のとおりに表現する．

```
<result> = cmovznz i1 <val3>, i64 0, i64 -1
```

`<val3>` が真（非ゼロ）なら `-1`（`0xFFFFFFFFFFFFFFFF`），偽（ゼロ）なら `0` が選ばれる．`sext` は符号を保った拡張を行う命令であり，`cmovznz` は条件に応じて二つの値を分岐なしで選択できるため，本写像は語彙の違いによる等価変換である．また，`i64` における `-1` の 2 の補数表現が `0xFFFFFFFFFFFFFFFF` である点とも一致する．さらに `cmovznz` は分岐を導入しないため，`sext` と同じタイミング特性を保つことができる．

4. 評価

本節では本提案方式が出力するコードの性能評価を行った結果について述べる．まず，評価に使った実装群の概要を示す．ついで，何をどのように比較したかという評価方法の詳細を述べる．最後に，実験環境を説明し，結果を提示する．

4.1 対象実装

本研究は 13 実装・計 25 関数（各実装の乗算・二乗算．ただし BLS12 は乗算のみ）を対象とする．概要は以下のとおりである．

- **Fiat Rust/Fiat C**：Fiat Cryptography が生成する Rust/C 実装 [14], [15] を本研究の LLVM ブリッジによる最適化対象とする．これらは Fiat JSON 前提の CryptOpt に取り込みやすく，Fiat Cryptography の形式的検証をそのまま活用し，Fiat Rust/Fiat C と CryptOpt 出力の同値性検査により正当性を継承できる．
- **AUCurves (BLS12)**：Fiat Cryptography と bedrock2 を基盤とする Rust 実装 [9], [13], [16] である．本稿では乗算を評価対象とする．
- **Curve25519-dalek/Rust-EC-secp256k1**：純 Rust 実装 [17], [18] を非 Fiat 系の代表として扱う．
- **OpenSSL**：Curve25519 は可搬 C 版と手書き最適化アセンブリの二系統があり，両者を比較対象とする．P448（素数 $p = 2^{448} - 2^{224} - 1$ 上の有限体を指し，Curve448 で用いられる）は可搬 C 版のみのため，CryptOpt 生成アセンブリと C コンパイル版を比較する [19]．

4.2 CryptOpt によるアセンブリ生成と本研究のベースライン生成

既存の CryptOpt 研究 [8] では，性能比較の基準（ベースライン）として，Fiat C を *GCC*/*Clang* でコンパイルして得た共有オブジェクト（`.so`）を用い，これに対して CryptOpt が生成した NASM アセンブリ（`.asm`）の実行速度を比較している（図 1）．しかし，ここには方法論上の問

題がある．CryptOpt の出力は NASM アセンブリ（.asm），計測の際は AssemblyLine によりオブジェクト（.o）にアセンブルされる一方で，Fiat C のベースラインは共有オブジェクト（.so）である．オブジェクト（.o）は典型的に静的リンクされるのに対し，共有オブジェクト（.so）は動的にロードされるため，この形式の不一致は厳密な同条件比較を誤らせる可能性がある．

この問題を解決するため，出力形式をオブジェクト（.o）に統一し，測定条件（リンカ，入力，CPU 設定）を揃えたうえで，コンパイラ生成コードと関数本体の命令列のみが異なる比較を実現する計測パイプラインを構築した．本パイプラインは，Rust または C を LLVM IR へコンパイル後，*llc -O3* で LLVM IR を GAS（GNU Assembler）アセンブリ（.asm）へ変換するところから始まる．これにより最上位最適化を適用したアセンブリを得る．CryptOpt の出力は NASM アセンブリ（.asm）であるため，コンパイラ生成の GAS アセンブリ（.asm）を NASM 形式へ変換するトランスレータも実装した．オペランド順序，レジスタ接頭辞を必要に応じて書き換えるが，機能的意味は変更しない．変換の詳細は appendix A.2 に示す．

最後に，自作の **Rust Assembly Measure** により，各アセンブリ（.asm）を *NASM*，*Clang* でアセンブルしオブジェクト（.o）を生成する．そして完全に同一条件下でそれらのオブジェクト（.o）をベンチマークする．

なお，CryptOpt で NASM アセンブリ（.asm）を生成する際のパラメータは原著 [8] と同一に設定している．

4.3 実験環境

実験は第 13 世代 Intel Core i7-1360P（Raptor Lake-P，8 コア/16 スレッド）上の Ubuntu 22.04.4 LTS で実施した．本評価では P-Core（Performance Core）上で計測を行った．計測環境の設定について以下の小節で述べる．

4.3.1 ノイズ低減

計測は単一の物理コアにピン留めし，SMT/Turbo Boost/動的周波数制御を無効化して周波数を固定，ASLR を停止し主要デーモンを停止・高頻度 IRQ を迂回させた．これにより，OS スケジューリング・割り込み・周波数変動・アドレス変動といった環境起因による計測値のばらつきを最小化した．

4.3.2 サイクル計測法

計時は適切なメモリバリア（mfence）とともに RDTSC を用い，バッチサイズは CryptOpt の方法論 [8] に従い動的に較正した．CryptOpt 生成物は Intel 表記（NASM 構文）であり *NASM* によりアセンブルする．一方，ベースラインのアセンブリは AT&T 表記（GAS のデフォルト）で得られるため，自動変換で Intel 表記へ機械的に変換し（詳細は appendix A.2），同一の *NASM* 呼び出しでアセンブルする．これより，被験要因は命令列そのものに限定され，アセンブラ差分は排除される．

4.3.3 統計的安定性のための反復計測

CryptOpt [8] のアルゴリズム単体では稀に周辺値から大きく乖離したサイクル数（外れ値/スパイク）が混入する．このような値を除去するため，CryptOpt のアルゴリズムを 31 回繰り返し，最後にそれらの中央値を最終性能値として採用し，結果を安定化させている．

4.4 結果

本節では，LLVM ブリッジを介して最適化した CryptOpt 生成 x86-64 アセンブリと，二種類のコンパイラ生成アセンブリとの性能比較を 表 1 に示す．指標は，ベースラインの中央値に対する CryptOpt 生成物の速度比（スピードアップ）である．

第 1 列「実装」は最適化対象の実装を示す．第 2 列「言語」は実装言語（Rust か C）を示す．第 3・4 列（および第 5・6 列）「GAS」「NASM」は，乗算（および二乗算）関数について，LLVM ブリッジ経由の CryptOpt 生成コードと汎用コンパイラ生成コード（ベースライン）を，それぞれ GAS 形式と NASM 形式で速度を比較した結果である．表中の数値は，ベースラインを 1.0 とした速度比を表す．1.0 より大きい値は本手法が高速であること，1.0 より小さい値はベースラインが高速であることを意味する．

また「—」は本パイプラインで未対応であることを示す．AUCurves (BLS12) 実装で観測された大幅な低下を含む性能差の要因については section 5 で議論する．

さらに，表 1 の OpenSSL の部分では，Curve25519 実装について，汎用コンパイラ生成コード（vs.OTS）および専門家による手書きアセンブリ（vs.Hand）との比較を示す．P448 には OpenSSL から手書きアセンブリが提供されていないため，比較は Curve25519 のみである．Curve25519 の乗算では，OpenSSL の専門家による手書きアセンブリ（vs.Hand）に対しても優位であり，GAS/NASM でそれぞれ 1.16 倍，1.26 倍高速である．

5. 議論

本節では，性能低下の潜在的要因ついて議論する．

5.1 レジスタ割り当てと BLS12 の性能

表 1 に示した性能比較によれば，我々の LLVM ブリッジを経て CryptOpt が最適化したコードは，AUCurves (BLS12) を例外として，通常のコンパイラ生成コードをほとんど同等かそれを上回った．生成されたアセンブリコードを詳細に分析したところ，特に AUCurves (BLS12) では，レジスタを空けるためレジスタからメモリへ値を退避させる mov 命令，いわゆるレジスタスピルが多数存在し，汎用コンパイラが生成するコードと比べてコードサイズが約 2 倍に膨らんでいることを見出した．

表 1 LLVM ブリッジ経由で CryptOpt により最適化したコードとコンパイラ生成コードの性能比較（速度比，P-Core）

		乗算		二乗算	
実装	**言語**	**GAS**	**NASM**	**GAS**	**NASM**
Curve25519-dalek	Rust	1.11	1.11	1.08	1.05
EC-Secp256k1	Rust	1.05	1.06	1.02	1.02
Fiat-Poly1305	Rust	1.10	1.10	0.94	0.94
Fiat-Curve25519	Rust	1.13	1.13	1.16	1.16
Fiat-NIST-P448	Rust	1.38	1.39	1.02	0.99
Fiat-Secp256k1	Rust	1.04	1.04	1.07	1.07
AUCurves-BLS12	Rust	0.61	0.59	—	—
Fiat-NIST-P448	C	1.39	1.39	0.99	0.99
Fiat-Curve25519	C	1.06	1.06	1.12	1.12
Fiat-Poly1305	C	1.05	1.05	1.06	1.06
Fiat-Secp256k1	C	1.05	1.05	1.09	1.09
OpenSSL 実装					
Curve25519	C (vs OTS)	1.09	1.09	1.13	1.13
Curve25519	C (vs Hand)	1.16	1.26	1.10	1.13
P448	C	1.24	1.23	1.00	1.00

注: —は本ワークフローがサポートしていない実装を示す．1.0 より大きい値は CryptOpt の方が高速であることを，1.0 より小さい値はベースラインの方が高速であることを示す．

現代的なコンパイラは詳細な生存区間解析を行い，干渉グラフを構築して変数の生存範囲を正確に捉え，スピルを最小化する．これに対して，我々の手法では LLVM IR から JSON への変換の段階で正確な生存範囲情報を失う．CryptOpt はこれを依存グラフのみに基づいて近似しながら扱うが，ある変数の生存範囲が不明確な場合，CryptOpt は後で必要になる可能性を潰さないよう，値を上書きしない保険としてレジスタをメモリにスピルする方を選ぶ．したがって，BLS12 のように規模が大きく複雑なルーチンでは，性能を大きく劣化させる．

5.2 LLVM IR に起因する制約

本研究のパイプラインでは，Rust/C 及び CryptOpt で実装されているが，LLVM プロジェクトが直接サポートしていない演算，具体的には `addcarryx` と `subborrow` が問題となる．`addcarryx` については，LLVM IR 上で 2 つの `add128` へと分解することが可能である．2 つの `add128` へ分解した後は，あらかじめ用意したテンプレートに従ってアセンブリへと変換する．しかしこの過程では，下位 64 ビット加算で生じたキャリービットを上位 64 ビット加算へ正しく伝搬させる処理を適切に扱えない．その結果，CryptOpt は `addcarryx` を正しく処理できず，これらの演算を含む Rust/C 実装は本研究のパイプラインを用いて CryptOpt で最適化することができない．

一方で `subborrow` については，128 ビット減算へ置き換える手法を用いることができない．理由は CryptOpt が符号なし 128 ビットの `sub` 演算をサポートしていないためであり，`subborrow` を `sub128` に分解して CryptOpt で扱うことができない．

ここでの制約をうける暗号プリミティブの具体例としては，NIST 曲線群（P224，P256，P384，P521）などが挙げられる．

6. まとめ

本論文は，暗号コードの自動最適化をより広い範囲へ拡張するフレームワークを提示する．CryptOpt エンジンを統合し，二つの主要な側面で強化した．

第一に，LLVM IR から CryptOpt の内部表現への変換を開発した．このブリッジにより，CryptOpt は既存の C および Rust 実装の広範な集合を最適化可能となり，従来の Fiat Cryptography への依存から解放される．

第二に，性能評価手法を改良した．新しい手法は，より正確で統計的に頑健な比較を提供し，外部ツールへの依存を排して，信頼性の高い結果を得る．

今後の課題として，汎用コンパイラに依存しない本ブリッジ向けのコンパイル経路を整備，CryptOpt がサポートする演算集合を拡張し，現代の暗号に必要なプリミティブを網羅することである．

参考文献

[1] Tung Chou. Sandy2x: new Curve25519 speed records. In *SAC*, pages 145–160, 2015.

[2] Tung Chou. QcBits: constant-time small-key code-based cryptography. In *CHES*, volume 9813, pages 280–300, 2016.

[3] Daniel J. Bernstein, Chitchanok Chuengsatiansup, Tanja Lange, and Peter Schwabe. Kummer strikes back: New DH speed records. In *ASIACRYPT*, pages 317–337, 2014.

[4] Matthias J. Kannwischer, Joost Rijneveld, and Peter Schwabe. Faster multiplication in $Z_{2^m}[x]$ on Cortex-M4 to speed up NIST PQC candidates. In *ACNS*, pages 281–301, 2019.

[5] Jean-Christophe Filliâtre and Andrei Paskevich. Why3 - where programs meet provers. In *ESOP*, pages 125–128, 2013.

[6] Yu-Fang Chen, Chang-Hong Hsu, Hsin-Hung Lin, Peter Schwabe, Ming-Hsien Tsai, Bow-Yaw Wang, Bo-Yin Yang, and Shang-Yi Yang. Verifying Curve25519 software. In *CCS*, pages 299–309, 2014.

[7] Andy Polyakov, Ming-Hsien Tsai, Bow-Yaw Wang, and Bo-Yin Yang. Verifying arithmetic assembly programs in cryptographic primitives (invited talk). In *CONCUR*, volume 118 of *LIPIcs*, pages 4:1–4:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.

[8] Joel Kuepper, Andres Erbsen, Jason Gross, Owen Conoly, Chuyue Sun, Samuel Tian, David Wu, Adam Chlipala, Chitchanok Chuengsatiansup, Daniel Genkin, Markus Wagner, and Yuval Yarom. CryptOpt: verified compilation with randomized program search for cryptographic primitives. *Proc. ACM Program. Lang.*, 7(PLDI):1268–1292, 2023.

[9] Andres Erbsen, Jade Philipoom, Jason Gross, Robert

Sloan, and Adam Chlipala. Simple high-level code for cryptographic arithmetic - with proofs, without compromises. In *IEEE Symposium on Security and Privacy*, pages 1202–1219. IEEE, 2019.

[10] 0xADE1A1DE. Measuresuite v2.2.2. `https://github.com/0xADE1A1DE/MeasureSuite`, 2023. Accessed: 2025-02-01.

[11] 0xADE1A1DE. Assemblyline v1.3.2. `https://github.com/0xADE1A1DE/AssemblyLine`, 2023. Accessed: 2025-02-01.

[12] llvm-admin team. The LLVM Compiler Infrastructure Project. `https://llvm.org/`, 2025. Accessed: 2025-05-07.

[13] AU-COBRA. Aucurves - high assurance cryptography by means of code synthesis. `https://github.com/AU-COBRA/AUCurves`, 2022. Accessed: 2025-02-01.

[14] The Fiat Cryptography Contributors. Fiat-Crypto rust bindings. `https://github.com/mit-plv/fiat-crypto/tree/main/rust`, 2023. commit `Commit 4e7dde9` (includes Curve25519, P448, Secp256k1, Poly1305).

[15] The Fiat Cryptography Contributors. Fiat-Crypto c bindings. `https://github.com/mit-plv/fiat-crypto/tree/main/c`, 2023. commit `Commit c79cf60` (includes Curve25519, P448, Secp256k1, Poly1305).

[16] mit plv. A work-in-progress language and compiler for verified low-level programming. `https://github.com/mit-plv/bedrock2`, 2018. Accessed: 2025-04-05.

[17] dalek-cryptography Developers. Dalek elliptic curve cryptography. `https://github.com/dalek-cryptography/curve25519-dalek`, 2023. Accessed: 2025-04-05.

[18] RustCrypto Developers. Rustcrypto. `https://github.com/RustCrypto`, 2020. Accessed: 2025-04-05.

[19] OpenSSL: Cryptography and SSL/TLS Toolkit. `https://github.com/openssl/openssl`. Commit snapshot accessed 2 July 2025. The evaluation uses the files `crypto/ec/curve25519.c` (portable C) and `crypto/ec/asm/x25519_x86_64.pl` (hand-optimised x86 – 64).

付　　録

A.1 JSON から Fiat JSON 類似 IR へ

第二段階では，Fiat JSON 類似 IR のための前処理が，フィルタ済みの JSON を形式的な Fiat JSON 類似 IR へ変換する．主眼は，定時間的なデータフローを保ったままメモリアクセスを抽象化する点にある．

A.1.1 ポインタ参照の配列アクセスへの解決

このスクリプトは低水準の `load`, `store`, `getelementptr` を取り込み，高水準の配列アクセスへ解決する．たとえば「引数配列の 5 番目の要素へのポインタを取り，その値をロードする」という意味の LLVM 命令列は，Fiat JSON 類似 IR の単一命令 `xN = arg1[4]` に変換される．これらの配列インデックスは元の getelementptr 命令中の定数整数から導かれるため，メモリアクセスの定時間性は保持される．この変換は新たなデータ依存を導入しない．ポインタ中心の表現を配列中心の表現へ置き換えるだけである．

A.1.2 算術の一対一対応

算術および論理の LLVM 命令（`add`, `mul`, `and`, `lshr` など）は，Fiat JSON 類似 IR の同等な意味を持つ命令へと一対一で写像する．これは直接変換であり，新たな可変時間動作を導入しない．

定時間性を侵すおそれのある構成要素をすべて除去し，意味保持的な抽象化を施すことで，最終的な Fiat JSON 類似 IR は定時間アルゴリズムの忠実な高水準表現となることをこのパイプラインは保証する．Fiat JSON 類似 IR を生成した後は，既存の CryptOpt [8] パイプラインがこれを CryptOpt IR へ変換し，それが CryptOpt への最終的な IR 入力となる．

A.2 ベースラインとアセンブリ形式

CryptOpt の出力とコンパイラのベースラインを公正に比較するために，コンパイラの GAS 構文を NASM 形式へ体系的に変換するスクリプトを実装する必要がある．このスクリプトは次のような主要な相違を処理する．

- **オペランド順序と接頭辞:** オペランドの入れ替え（AT&T `src,dst` → Intel `dst,src`），レジスタ接頭辞「`%`」の除去（`%rax` → `rax`），即値接頭辞「`$`」の除去（`$42` → `42`）
- **ニーモニックとアドレッシング:** 命令名の正規化（`movabs` → `mov`），メモリオペランドの変換（`8(%rax)` → `QWORD[rax+8]`）
- **アセンブラ疑似命令:** 主要ディレクティブの写像（`.globl` → `global`, `.align N` → `ALIGN N`）、GNU 固有メタデータの除去（`.type`, `.size`, `# %bb.*`）

変換の正当性は差分テストにより検証した．まず LLVM IR から *llc* を用いて基準となるオブジェクト（`.o`）を直接生成する．次に，この基準オブジェクト（`.o`），変換後の NASM オブジェクト（`.o`），および CryptOpt が生成したオブジェクト（`.o`）を，同一のランダム入力を多数回にわたり与えて実行し，それらの出力が完全に一致することを検証する．不一致は一切観測されず，GAS と NASM のベースライン間で見られる計時差（通常 5% 未満）は，アセンブラ固有のレイアウト最適化に起因すると考えられる．